NG RP // INTERVIEW SERIES

JUNIOR → SENIOR

VOL 01 · FRONTEND INTERVIEW

# Angular Interview Roadmap.

## Junior → Senior

For Angular developers preparing for product-company interviews — concepts, patterns, and the answers that get you hired.

Signals | Change Detection | RxJS | NgRx | SSR | Performance | Testing

130+ PAGES

90+ QUESTIONS

8 MB PDF SIZE

AUTHORED BY
Rutik Pimpale

UPDATED
Apr 2025

NAVIGATION

# Table of Contents

CLICK ANY ENTRY TO JUMP TO THE SECTION

## 07 TOPIC 7: FORMS (TEMPLATE-DRIVEN + REACTIVE)

- Q1: What is the difference between Template-driven and Reactive Forms? ★
- Q2: How do you create a Reactive Form? ★
- Q3: What is FormGroup, FormControl, and FormArray?
- Q4: How do you handle form validation? ★
- Q5: What is ngModel and two-way binding?
- Q6: How do Angular Forms work internally?

## 08 TOPIC 8: HTTP & API HANDLING

- Q1: How do you make HTTP requests in Angular? ★
- Q2: What are HTTP Interceptors? ★
- Q3: How do you handle errors in HTTP calls? ★
- Q4: What is the difference between REST API and SOAP API?
- Q5: How do you implement caching in Angular?
- Q6: How do you handle multiple API calls?

## 09 TOPIC 9: RXJS BASICS

- Q1: What is RxJS and why is it used in Angular? ★
- Q2: What is the difference between Observable and Promise? ★
- Q3: What are commonly used RxJS operators? ★
- Q4: Explain switchMap with a real example. ★
- Q5: What is Subject and BehaviorSubject? ★
- Q6: When should you unsubscribe from Observables and why?
- Q7: How do you optimize API calls with RxJS?

## 10 TOPIC 10: LIFECYCLE HOOKS

- Q1: What are Lifecycle Hooks? Name the important ones. ★
- Q2: What is the difference between constructor and ngOnInit? ★
- Q3: What is ngOnDestroy? Why is it important? ★
- Q4: What is ngOnChanges? When does it fire?
- Q5: What is ngAfterViewInit?

## 11 TOPIC 11: STATE MANAGEMENT (BASIC)

- Q1: What is State Management? Why do we need it? ★
- Q2: How do you manage state using Services? ★
- Q3: What is NgRx? When would you use it?
- Q4: What are Angular Signals? (Angular 16+)

## 12 TOPIC 12: PERFORMANCE OPTIMIZATION

- Q1: How do you improve Angular app performance? ★
- Q2: What is OnPush Change Detection? How does it help? ★
- Q3: How do you handle memory leaks in Angular? ★
- Q4: What is Virtual Scrolling?
- Q5: What is Tree Shaking?
- Q6: What is Change Detection and how does it work internally?
- Q7: What is Zone.js and what role does it play in Angular?
- Q8: How do you reduce the bundle size of an Angular app?

## 13 TOPIC 13: ANGULAR INTERVIEW SCENARIO QUESTIONS

- Q1: Tell me about a challenging problem you solved in Angular. ★
- Q2: How do you structure a large Angular application? ★
- Q3: How do you handle authentication in Angular? ★
- Q4: How do you handle errors globally in your app?
- Q5: Explain a situation where you optimized performance. ★
- Q6: What is the difference between ViewChild and ContentChild?
- Q7: What are Standalone Components? (Angular 15+)
- Q8: What is the new Control Flow syntax in Angular 17+?
- Q9: How do you improve Angular app scalability?

## 14 QUICK REVISION CHEAT SHEET

## 15 INTERVIEW TIPS

# TOPIC 1: ANGULAR BASICS

## Q1: What is Angular? (IMPORTANT)

**A:** Angular is a **TypeScript-based frontend framework developed by Google** for building **single-page applications (SPAs)**. Unlike libraries, Angular is a **complete solution** — it comes with built-in support for **routing, forms, HTTP handling, dependency injection, and state management**, so we don't need to rely on third-party packages for basic functionality.

What makes Angular powerful is its **opinionated architecture** — it enforces a clean structure using **modules, components, and services**, which is very helpful when working in teams or on large-scale projects.

**Real Example:** In my project, we built a complete admin dashboard as a SPA using Angular. The entire application — user management, reports, settings — runs without a single page reload, and the **modular structure** made it easy for 4 developers to work on different features simultaneously.

Angular is the .

- *What is the difference between Angular and AngularJS?* — AngularJS (v1) was the old version built on **JavaScript** with **$scope-based two-way binding**. Angular (v2+) is a **complete rewrite** using **TypeScript**, **component-based architecture**, and is significantly **faster and more maintainable**.
- *Why Angular over React?* — Angular is a **full-fledged framework** with routing, forms, DI, and HTTP built-in. React is a **UI library** — you need to add React Router, Redux, Axios, etc. separately. For large enterprise apps, Angular gives a **consistent structure out of the box**.

## Q2: What is TypeScript and why does Angular use it? (IMPORTANT)

**A:** TypeScript is a **strongly-typed superset of JavaScript** developed by Microsoft. Angular chose TypeScript as its primary language because it provides **compile-time type checking**, **better IDE intelligence** (autocomplete, refactoring, error highlighting), and makes **large codebases much easier to maintain and debug**.

The biggest advantage I've experienced is that TypeScript catches bugs . For example, if I accidentally pass a string where a number is expected, my IDE immediately shows a red underline — I don't have to wait until the app runs and crashes.

**Real Example:** In my project, we had a user object with 15+ properties. With TypeScript **interfaces**, every developer knew exactly what properties were available, and the IDE auto-suggested them — this **reduced bugs by a significant margin** and made onboarding new developers faster.

TypeScript brings to Angular development.

- *Can we use Angular without TypeScript?* — Technically yes with plain JavaScript, but it's **strongly not recommended**. You lose type safety, decorator support, and the powerful tooling that makes Angular productive.

---

## Q3: What is the difference between JIT and AOT compilation? (IMPORTANT)

**A:** **JIT (Just-in-Time)** compiles the Angular application **in the browser at runtime** — this is what happens during development when we run `ng serve`. **AOT (Ahead-of-Time)** compiles everything **during the build process itself** — this is used for production builds with `ng build`.

AOT is preferred for production because the browser receives **pre-compiled code**, which means **faster rendering**, **smaller bundle size** (Angular compiler is not shipped), and **template errors are caught at build time** rather than failing silently in production.

**Real Example:** When we switched from JIT to AOT in our project, the initial load time **dropped noticeably** because the browser didn't have to compile templates anymore — it just rendered the pre-compiled output directly.

AOT gives you .

- *Which one does* `ng serve` *use?* — JIT by default (for faster rebuilds during development).
- *Which one does* `ng build` *use?* — AOT by default since Angular 9+ (Ivy compiler).

---

## Q4: What are the main building blocks of Angular?

**A:** Angular has a well-defined set of **building blocks** that work together to create a structured application. Each one has a **specific responsibility**, which keeps the codebase organized and maintainable.

- **Components** — the core UI building block, each controlling a section of the screen
- **Templates** — the HTML view attached to a component
- **Modules** — organize related code into cohesive blocks (though **standalone components** are replacing this in newer Angular)
- **Services** — hold **business logic, API calls, and shared data** — keep components clean
- **Directives** — add custom behavior to DOM elements (`*ngIf`, `*ngFor`, `ngClass`)
- **Pipes** — transform displayed data in templates (like formatting dates or currency)
- **Routing** — handles navigation between different views without page reload

**Real Example:** In any feature I build — say a "User Management" module — I create a **component** for UI, a **service** for API calls, use **pipes** to format dates, **directives** to conditionally show elements, and **routing** to navigate between list and detail views.

These building blocks enforce **separation of concerns**, which is key for **scalable architecture**.

- *Which building block do you use most?* — **Components and Services** — almost every feature involves creating a component for UI and a service for data/logic.

---

## Q5: What is Angular CLI? Name some common commands.

**A:** Angular CLI is a **powerful command-line tool** that automates repetitive tasks like project setup, code generation, building, testing, and deployment. It enforces **best practices** and consistent project structure across teams.

I use it — from generating components to building production bundles. It saves a lot of manual configuration and boilerplate work.

| Command | Purpose |
|---|---|
| `ng new app-name` | Create new project with full setup |
| `ng serve` | Run dev server with **live reload** |
| `ng g c component-name` | Generate component with files |
| `ng g s service-name` | Generate service |
| `ng g m module-name` | Generate module |
| `ng build` | Build optimized production bundle |
| `ng test` | Run unit tests with Karma |

**Real Example:** When I need a new feature, I just run `ng g c features/user-list` and Angular CLI creates the component with its HTML, CSS, spec file, and even updates the module — all in one command.

Angular CLI is an for any Angular developer.

- *What does* `ng g c --skip-tests` *do?* — Creates the component **without** the `.spec.ts` test file — useful for quick prototyping.

## Q6: What is the difference between var, let, and const?

**A:** These are the three ways to declare variables in JavaScript/TypeScript. Since Angular runs on TypeScript, understanding this is fundamental. The key differences are **scope**, **reassignment**, and **hoisting behavior**.

| Feature | `var` | `let` | `const` |
|---|---|---|---|
| **Scope** | Function-scoped | Block-scoped `{ }` | Block-scoped `{ }` |
| **Reassignment** | Allowed | Allowed | **Not allowed** |
| **Redeclaration** | Allowed | Not allowed | Not allowed |
| **Hoisting** | Hoisted (initialized as `undefined`) | Hoisted but not initialized (TDZ) | Hoisted but not initialized (TDZ) |
| **Use in Angular** | Avoid | For values that change | **Default choice** |

```
// var — function-scoped, can leak out of blocks
if (true) { var a = 10; }
console.log(a); // 10 — leaked out!

// let — block-scoped, safe
if (true) { let b = 20; }
console.log(b); // ReferenceError

// const — block-scoped + immutable reference
const user = { name: 'Rutik' };
user.name = 'Angular';  // OK — mutating properties is fine
user = {};              // Error — can't reassign the reference
```

**Real Example:** In my Angular project, I use `const` by default for everything — services injected via DI, arrays, config objects. I use `let` only for loop counters or values that genuinely need reassignment. I **never use** `var` — it causes scope leaks and bugs that are hard to trace.

- *Does `const` make an object immutable?* — No. `const` only prevents **reassigning the variable**. Object properties can still change. For true immutability, use `Object.freeze()` or `readonly` TypeScript modifiers.
- *What is TDZ (Temporal Dead Zone)?* — The period between entering a block and the `let`/`const` declaration — accessing the variable there throws `ReferenceError`. This helps catch bugs early.

# TOPIC 2: COMPONENTS & DATA BINDING

## Q1: What is a Component in Angular? (IMPORTANT)

**A:** A component is the **fundamental building block** of any Angular application. It controls a **specific part of the UI** and encapsulates its own **logic, template, and styling**. Every Angular app is essentially a **tree of components** — starting from the root `AppComponent`.

Each component has 3 parts:

- **TypeScript class** — holds the data and logic
- **HTML template** — defines what the user sees
- **CSS/SCSS** — scoped styling for that component only

We define it using the `@Component` decorator, which provides metadata like the **selector** (custom HTML tag), **template**, and **styles**.

```
@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html',
  styleUrls: ['./hello.component.css']
})
export class HelloComponent {
  message = 'Hello World!';
}
```

**Real Example:** In my project, the dashboard page was a single component composed of child components — `HeaderComponent`, `SidebarComponent`, `ChartComponent`, `TableComponent`. Each handled its own UI and logic independently, making the code **modular and easy to maintain**.

Components enable .

- *What is a selector?* — It's a **custom HTML tag** like `<app-hello>` that we use to embed this component anywhere in other templates.
- *What is the difference between* `templateUrl` *and* `template`*?* — `templateUrl` links to an **external HTML file**. `template` allows writing **inline HTML** directly inside the decorator — useful for very small templates.

## Q2: What is Data Binding? What are its types? (IMPORTANT)

**A:** Data binding is the mechanism that creates a **live connection between the component's TypeScript class and its HTML template**. It's what makes Angular apps dynamic — when data changes, the view updates automatically. Angular supports **4 types of data binding**:

| Type | Syntax | Direction | Use Case |
|---|---|---|---|
| **Interpolation** | `{{ value }}` | Component → View | Display dynamic text |
| **Property Binding** | `[property]="value"` | Component → View | Bind element properties |
| **Event Binding** | `(event)="handler()"` | View → Component | Handle user actions |
| **Two-way Binding** | `[(ngModel)]="value"` | Both ways | Sync form inputs |

**Real Example:** In a search feature, I use `[(ngModel)]` on the search input so the variable updates as the user types, and I can **filter results in real-time** without any extra code for syncing.

Data binding is the — it eliminates manual DOM manipulation entirely.

- *What module is needed for ngModel?* — `FormsModule` must be imported in the module.
- *What is the difference between property binding and interpolation?* — Interpolation only works for **string values**. Property binding can bind **any type** — boolean, object, array. For example, `[disabled]="isLoading"` requires property binding because it's a boolean.

## Q3: How do you pass data between components? (IMPORTANT)

**A:** This is one of the most common tasks in Angular. There are **3 main approaches**, and I choose based on the **relationship between components**:

The parent binds data to a child property using property binding.

```
// Child component
@Input() userName: string;

// Parent template
<app-child [userName]="parentName"></app-child>
```

The child emits events that the parent listens to.

```ts
// Child component
@Output() dataEmitter = new EventEmitter<string>();
sendData() {
  this.dataEmitter.emit('Hello from child!');
}

// Parent template
<app-child (dataEmitter)="receiveData($event)"></app-child>
```

**3. Between unrelated components — Shared Service with BehaviorSubject** When components have no parent-child relation, a shared service acts as the **communication bridge**.

**Real Example:** In my project, the **sidebar component** emits the selected menu item via a shared service, and the **main content area** subscribes to it and loads the corresponding view — both components are siblings with no direct relationship.

Choosing the right approach depends on .

- *Why use BehaviorSubject instead of Subject?* — BehaviorSubject **holds the last emitted value**, so even if a component subscribes **after** the data was sent, it still receives the most recent value immediately.
- *Can we use @Input() for deeply nested components?* — We can, but it creates **prop drilling** (passing data through multiple levels). A shared service is **cleaner and more scalable** for deep nesting.

---

## Q4: What is @Component decorator?

**A:** `@Component()` is a **TypeScript decorator** that marks a class as an Angular component and provides the **metadata** Angular needs to create and render it. Without this decorator, Angular has no way of knowing that a class should behave as a component.

Key metadata properties:

- `selector` — the custom HTML tag name (e.g., `app-user`)
- `templateUrl` — path to the external HTML file
- `styleUrls` — array of CSS/SCSS files for scoped styling
- `providers` — component-level service instances
- `changeDetection` — strategy for detecting changes (Default or OnPush)
- `standalone` — marks as standalone component (Angular 15+)

**Real Example:** When I set `changeDetection: ChangeDetectionStrategy.OnPush` in the decorator, Angular only re-checks this component when its `@Input()` reference changes — this **significantly**

**improved performance** in our data-heavy table component.

The `@Component` decorator is what **transforms a plain TypeScript class into a fully functional Angular component**.

---

## Q5: What is String Interpolation?

**A:** String interpolation is the simplest form of data binding in Angular. It uses **double curly braces** `{{ }}` to display component data directly inside the HTML template. It's a **one-way binding** — data flows from the TypeScript class to the view only.

```
<h1>Welcome, {{ username }}!</h1>
<p>Total: {{ price * quantity }}</p>
<p>Status: {{ isActive ? 'Active' : 'Inactive' }}</p>
```

We can use **simple expressions** inside interpolation — variables, arithmetic, ternary operators, and method calls. But **complex logic** like if-else blocks, assignments, or loops is not allowed.

**Real Example:** In my project's header component, I display `{{ currentUser.name }}` and `{{ notificationCount }}` using interpolation — it's the quickest way to show dynamic data in templates.

Interpolation is .

- *Can we call a function inside interpolation?* — Yes, like `{{ getTotal() }}`, but it's a **performance anti-pattern** because the function gets called on **every change detection cycle**. Better to use a **variable or pure pipe** instead.

---

## Q6: What are Smart and Dumb Components?

**A:** This is a popular architectural pattern in Angular (borrowed from React) that **separates responsibilities** between components for better reusability, testability, and maintainability.

| Aspect | Smart (Container) Components | Dumb (Presentational) Components |
|---|---|---|
| **Purpose** | Manage data and logic | Display data and emit events |
| **Services** | Inject services, call APIs | No services injected |
| **Inputs/Outputs** | Rarely used | Heavy use of `@Input()` and `@Output()` |
| **Routing** | Usually routed components | Typically children only |
| **Reusability** | Low (feature-specific) | High (reusable everywhere) |
| **Change Detection** | Default | `OnPush` (performance boost) |

**Real Example:** In my project's user management feature, `UserListContainerComponent` (smart) calls `UserService.getUsers()`, handles pagination state, and passes data down. `UserCardComponent` (dumb) just receives a `user` input and emits `edit` / `delete` events. The same `UserCardComponent` is reused in the dashboard, profile page, and search results — because it doesn't know where its data comes from.

```
// Smart — owns data + logic
@Component({ selector: 'app-user-list-container', ... })
export class UserListContainerComponent {
  users$ = this.userService.getUsers();
  constructor(private userService: UserService) {}
  onDelete(id: string) { this.userService.delete(id); }
}

// Dumb — pure input/output
@Component({
  selector: 'app-user-card',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div>{{ user.name }}</div>
    <button (click)="delete.emit(user.id)">Delete</button>
  `
})
export class UserCardComponent {
  @Input() user!: User;
  @Output() delete = new EventEmitter<string>();
}
```

- **Reusability** — dumb components work in any context
- **Testability** — dumb components test with simple inputs, no service mocks
- **Performance** — dumb components easily use `OnPush`
- **Maintainability** — clear boundaries of "who does what"

- *Can dumb components have their own state?* — Yes, **UI-only state** like "is dropdown open" is fine. But **business state** (user lists, filters, auth) belongs in smart components or services.

---

# TOPIC 3: DIRECTIVES

### Q1: What are Directives in Angular? What are the types? (IMPORTANT)

**A:** Directives are **special instructions that tell Angular how to modify or manipulate DOM elements**. They are one of the most powerful features of Angular, allowing us to add dynamic behavior to templates. There are **3 types**:

— Components are technically directives with their own template. Every component we create is a directive.

**2. Structural Directives** — These **change the DOM structure** by adding, removing, or replacing elements:

- `*ngIf` — conditionally add/remove an element
- `*ngFor` — iterate over a collection and render elements
- `*ngSwitch` — switch between multiple views based on a condition

**3. Attribute Directives** — These **change the appearance or behavior** of an existing element:

- `ngClass` — dynamically add/remove CSS classes
- `ngStyle` — dynamically apply inline styles

**Real Example:** In my project, I use `*ngIf` to show a **loading spinner** while the API is fetching data, and once data arrives, `*ngFor` renders the list. I use `ngClass` to highlight the **active row** in a table when the user clicks on it.

Directives let you without writing manual DOM manipulation code.

- *Why do structural directives have a `*` (asterisk)?* — The `*` is **syntactic sugar**. Behind the scenes, Angular converts it into an `<ng-template>` with the directive applied. It's just a shorthand for cleaner syntax.

---

### Q2: What is the difference between *ngIf and [hidden]? (IMPORTANT)

**A:** Both hide elements from the user, but they work **completely differently under the hood**. `*ngIf` **physically removes or adds** the element from the DOM, while `[hidden]` just applies `display: none`

— the element **stays in the DOM** but becomes invisible.

| Feature | `*ngIf` | `[hidden]` |
|---|---|---|
| How it works | **Removes/adds** element from DOM | **Hides** with CSS `display:none` |
| DOM impact | Element doesn't exist when false | Element always exists in DOM |
| Performance | Better for heavy components | Better for frequent toggling |
| Lifecycle hooks | Triggers `ngOnInit` / `ngOnDestroy` each time | No lifecycle impact |

**Real Example:** For a **modal popup** with complex form logic inside, I use `*ngIf` — no point keeping it in the DOM when it's closed. But for a **dropdown menu** that the user toggles frequently, I use `[hidden]` to avoid re-creating it every time.

The choice depends on whether you want to **destroy and recreate** or just **visually toggle** the element.

- *When would `[hidden]` be a bad choice?* — When the hidden element has **active subscriptions, timers, or heavy computations** running — it wastes resources because the component is still alive in the DOM.

---

## Q3: How does *ngFor work? What is trackBy? (IMPORTANT)

**A:** `*ngFor` is a **structural directive** that iterates over an array and renders a template block for each item. It also gives access to useful variables like `index`, `first`, `last`, `even`, and `odd`.

```
<div *ngFor="let user of users; let i = index">
  {{ i + 1 }}. {{ user.name }}
</div>
```

`trackBy` is a **performance optimization** that tells Angular how to **uniquely identify each item** in the list. Without it, Angular has no way to know which items changed — so it **destroys and recreates the entire DOM list** on every update. With `trackBy`, Angular only **updates the specific items that actually changed**.

```
trackById(index: number, user: any) {
  return user.id;
}
```

```
<div *ngFor="let user of users; trackBy: trackById">
```

**Real Example:** I had a real-time user list refreshing every 10 seconds from an API. Without `trackBy`, the entire table **flickered on each refresh**. After adding `trackBy: trackById`, only the changed rows updated — **smooth and flicker-free**.

`trackBy` is a **must-have for any list that updates dynamically** — it prevents unnecessary DOM manipulation.

- *What happens if you don't use trackBy with large lists?* — Angular destroys and recreates **all DOM elements** even if only one item changed — causes **poor performance, flickering, and loss of element state** (like input focus).

---

## Q4: What is ngClass and ngStyle?

**A:** Both are **attribute directives** used for **dynamic styling**, but they serve different purposes. `ngClass` works with **CSS classes** while `ngStyle` works with **inline styles**.

**ngClass** — dynamically adds or removes CSS **classes** based on conditions:

```
<div [ngClass]="{ 'active': isActive, 'disabled': isDisabled, 'highlight': isSelected }">
```

**ngStyle** — dynamically applies **inline styles**:

```
<div [ngStyle]="{ 'color': isError ? 'red' : 'green', 'font-size': fontSize + 'px' }">
```

**Real Example:** In my project, I use `ngClass` to toggle an `active` class on the currently selected navigation item, and `ngStyle` to dynamically set the **width of a progress bar** based on a percentage value from the API.

**When to use which:** Use `ngClass` when you have **predefined CSS classes** ready. Use `ngStyle` when you need to **compute style values dynamically** at runtime (like colors from a theme API).

---

## Q5: What is ng-template, ng-container, and ng-content?

**A:** These three are **Angular-specific elements** that don't render any actual DOM — they serve special structural purposes:

- `<ng-template>` — Defines a **lazy template block** that Angular **doesn't render by default**. It only renders when explicitly told to — via structural directives, `*ngIf else`, or `ngTemplateOutlet`.

```
<div *ngIf="isLoaded; else loading">Data loaded!</div>
<ng-template #loading><p>Loading...</p></ng-template>
```

- `<ng-container>` — A **logical grouping element** that **adds zero extra DOM nodes**. Perfect when you need to apply a structural directive without adding a wrapper `div` that breaks your CSS layout.

```
<ng-container *ngIf="isLoggedIn">
  <p>Welcome!</p>
  <p>Your dashboard</p>
</ng-container>
```

- `<ng-content>` — Enables **content projection** (similar to React's `children`). It allows a parent component to **inject custom content** into a child component's template.

```
<!-- Child template -->
<div class="card">
  <ng-content></ng-content>
</div>

<!-- Parent using child -->
<app-card>
  <p>This content goes inside the card!</p>
</app-card>
```

**Real Example:** In my project, I built a **reusable card component** using `<ng-content>` — different pages inject their own content into the same card layout. I used `<ng-container>` to apply `*ngIf` on a group of elements without adding unnecessary `div` wrappers.

These three are .

- *What is multi-slot content projection?* — Using `<ng-content select=".header">` and `<ng-content select=".body">` to project content into **specific named slots** based on CSS selectors.

# TOPIC 4: PIPES

## Q1: What is a Pipe in Angular? (IMPORTANT)

**A:** A Pipe is a **template-level data transformer** — it takes a value, processes it, and returns the formatted output **without modifying the original data** in the component. We apply pipes using the **`|` (pipe) symbol** directly in HTML templates.

```
{{ birthday | date:'dd/MM/yyyy' }}
{{ price | currency:'INR' }}
{{ name | uppercase }}
{{ longText | slice:0:50 }}
```

**Common built-in pipes:** `date`, `currency`, `uppercase`, `lowercase`, `titlecase`, `slice`, `json`, `async`, `percent`, `number`

**Real Example:** In my project, API dates come in ISO format like `2024-01-15T10:30:00Z`. Instead of writing formatting logic in the component, I simply use `{{ createdAt | date:'dd MMM yyyy' }}` in the template — it displays as `15 Jan 2024`. Clean and reusable.

Pipes keep **display logic in the template** where it belongs, making components **leaner and more focused on business logic**.

- *What is the difference between pure and impure pipes?* — **Pure pipes** execute only when the **input reference changes** — very efficient. **Impure pipes** execute on **every change detection cycle** — powerful but can impact performance on large datasets.

## Q2: How do you create a Custom Pipe? (IMPORTANT)

**A:** Creating a custom pipe involves 3 simple steps: create a class that implements `PipeTransform`, add the `@Pipe` decorator with a **name**, and implement the `transform()` method with your formatting logic.

```typescript
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'capitalize' })
export class CapitalizePipe implements PipeTransform {
  transform(value: string): string {
    if (!value) return '';
    return value.charAt(0).toUpperCase() + value.slice(1);
  }
}
```

```typescript
{{ 'hello world' | capitalize }}
<!— Output: Hello world —>
```

**Real Example:** I created a `timeAgo` custom pipe that converts timestamps into human-readable format — `"2 hours ago"`, `"3 days ago"`. I used it across the **notifications list, activity feed, and comment sections** — one pipe, reused everywhere.

Custom pipes are perfect for that you need across multiple templates.

- *How do you pass parameters to a custom pipe?* — Add extra arguments to the `transform()` method: `transform(value: string, limit: number)` and use as `{{ text | truncate:50 }}`.

---

## Q3: What is the difference between Pure and Impure Pipes?

**A:** This is an important performance concept. **Pure pipes** (default) only re-execute when the **input value reference changes** — Angular caches the result and reuses it. **Impure pipes** re-execute on **every single change detection cycle**, regardless of whether the input changed or not.

| Feature | Pure Pipe | Impure Pipe |
|---|---|---|
| Default | Yes (default behavior) | Must set `pure: false` |
| When it runs | Only when input **reference** changes | On **every** change detection |
| Performance | Highly efficient | Can be expensive |
| Use case | Simple transforms (format, uppercase) | Filtering/sorting arrays that mutate |

```typescript
@Pipe({ name: 'filter', pure: false })
```

**Real Example:** I once created a filter pipe for a table and kept it as pure. It wasn't updating when array items were modified. Switching to `pure: false` fixed it — but I quickly realized the **performance cost** on a 500-row table. So I refactored to use **immutable data patterns** with a pure pipe instead.

**Rule of thumb:** Always prefer **pure pipes** and use **immutable data** — only use impure pipes as a last resort.

---

## Q4: What is the Async Pipe? (IMPORTANT)

**A:** The `async` pipe is one of the **most powerful built-in pipes** in Angular. It **subscribes to an Observable or Promise** directly in the template, renders the latest emitted value, and **automatically unsubscribes** when the component is destroyed. This means **zero manual subscription management and zero memory leak risk**.

```
// Component — no subscribe() needed!
users$ = this.http.get<User[]>('/api/users');
```

```
<!— Template —>
<div *ngFor="let user of users$ | async">
  {{ user.name }}
</div>
```

- No manual `.subscribe()` and `.unsubscribe()` boilerplate
- **Prevents memory leaks automatically**
- Works perfectly with **OnPush change detection**
- Results in **cleaner, more declarative code**

**Real Example:** In my project, I switched from manual subscriptions to `async` pipe across most components — it reduced our code by roughly 30% and eliminated several memory leak issues we were debugging.

The async pipe is the .

- *When would you NOT use async pipe?* — When I need to **manipulate or combine data** before displaying, or when the same observable is used **multiple times** in the template (creates multiple subscriptions). For the latter, I use `*ngIf` with `as` to solve it:

```
<ng-container *ngIf="users$ | async as users">
  <p>Total: {{ users.length }}</p>
  <div *ngFor="let user of users">{{ user.name }}</div>
</ng-container>
```

# TOPIC 5: SERVICES & DEPENDENCY INJECTION

## Q1: What is a Service in Angular? (IMPORTANT)

**A:** A service is a **reusable TypeScript class** that holds **business logic, API communication, or shared data** — basically anything that's not directly related to the UI. The idea is to keep components **thin and focused only on presentation**, while services handle the heavy lifting behind the scenes.

```
@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}
```

**Real Example:** In my project, I have clearly separated services — `AuthService` handles login/logout/token management, `UserService` handles user CRUD operations, `NotificationService` manages toast messages. Each service has a **single responsibility**, making them easy to test and maintain.

Services enforce the **single responsibility principle** and are the backbone of **clean Angular architecture**.

- *Can a service inject another service?* — Absolutely! As long as both have `@Injectable()`. For example, my `UserService` injects `HttpClient` for API calls and `LoggerService` for tracking errors — this is a very common pattern.

## Q2: What is Dependency Injection (DI)? (IMPORTANT)

**A:** Dependency Injection is a **design pattern** where Angular's DI framework **automatically creates and provides** the dependencies a class needs, instead of the class creating them manually. This makes our code **loosely coupled, highly testable, and easily maintainable**.

```
// Angular injects UserService automatically — no 'new' keyword needed
constructor(private userService: UserService) {}
```

I never write `new UserService()` anywhere in my code. Angular's **injector** creates the instance, manages its lifecycle, and provides the **same singleton instance** wherever it's needed. During testing, I can easily **swap the real service with a mock** — that's the power of DI.

**Real Example:** In my project, by using DI, I can inject a `MockUserService` during unit tests that returns fake data — no actual API calls, tests run **fast and reliably**.

DI is what makes Angular apps .

- *What is an Injector?* — The injector is the **engine that creates and manages service instances**. Angular has a **hierarchical injector system** — root injector (app-wide), module injectors, and component injectors — each level can have its own instance.

## Q3: What is @Injectable()? What does providedIn: 'root' mean? (IMPORTANT)

**A:** `@Injectable()` is a decorator that tells Angular this class **can participate in the dependency injection system** — meaning it can be injected into other classes, and it can also have dependencies injected into itself (like `HttpClient`).

`providedIn: 'root'` is the recommended way to register a service. It means:

- The service is a **singleton** — only one instance exists for the entire app
- It's **tree-shakable** — if no component uses it, Angular removes it from the production bundle automatically
- **No manual registration** needed in any `providers[]` array

| Value | Meaning |
|---|---|
| `'root'` | Single instance for the entire app (most common) |
| `'any'` | New instance for every lazy-loaded module |
| `SomeModule` | Scoped to a specific module |

**Real Example:** My `AuthService` uses `providedIn: 'root'` because the same login state must be accessible everywhere in the app. But for a `FormHelperService` that only a specific feature module needs, I provide it at the **component level** to get a fresh instance per component.

`providedIn: 'root'` is the **modern, recommended way** to register Angular services.

- *What happens if I provide a service in a component's providers array?* — A **new instance** is created specifically for that component and its children. Each component tree gets its own isolated copy of the service.

---

### Q4: What is the difference between providedIn: 'root' and adding to providers array?

**A:** Both register a service for dependency injection, but they differ in **scope, tree-shaking, and recommended usage**:

| Feature | `providedIn: 'root'` | `providers: []` in module |
|---|---|---|
| **Singleton** | Yes, app-wide automatically | Yes, within that module's scope |
| **Tree-shakable** | Yes — unused services removed from bundle | No — always included |
| **Registration** | Automatic (in the service itself) | Manual (in module/component) |
| **Recommended** | Yes — Angular's official recommendation | Only for special scoping needs |

**Real Example:** In my project, all shared services like `AuthService`, `ApiService`, `NotificationService` use `providedIn: 'root'`. But for a `FormValidationService` that needs a **separate instance per feature module**, I register it in the module's `providers[]` array.

**Best Practice:** Default to `providedIn: 'root'`. Only use `providers[]` when you **intentionally need multiple instances** or module-scoped services.

## Q5: How do you share data between unrelated components using a Service?

**A:** The cleanest approach is using a **shared service with BehaviorSubject**. The service acts as a **centralized communication hub** — one component pushes data into it, and any other component can subscribe to receive updates in real-time.

```
@Injectable({ providedIn: 'root' })
export class SharedDataService {
  private messageSource = new BehaviorSubject<string>('default');
  currentMessage$ = this.messageSource.asObservable();

  updateMessage(message: string) {
    this.messageSource.next(message);
  }
}
```

```
// Component A – sends data
this.sharedService.updateMessage('Hello from A!');

// Component B – receives data
this.sharedService.currentMessage$.subscribe(msg => {
  this.message = msg;
});
```

**Why BehaviorSubject over Subject?** — BehaviorSubject **retains the last emitted value**. So even if Component B subscribes **5 seconds after** Component A sent the data, B still gets the latest value immediately. With plain Subject, it would have missed it.

**Real Example:** In my project, the **sidebar navigation** and **breadcrumb component** are siblings — when the user selects a menu item in the sidebar, the breadcrumb updates automatically through a shared `NavigationService` using BehaviorSubject.

This pattern is the and works perfectly for most applications.

- *Why not just use @Input/@Output?* — Those only work for **parent-child** relationships. For **sibling or completely unrelated components**, a shared service is the cleanest and most scalable approach.

## Q6: How does Angular's Dependency Injection Hierarchy work?

**A:** Angular's DI system is **hierarchical** — it mirrors the component tree. When a component requests a dependency, Angular walks **up the injector tree** until it finds a provider. This lets us **scope services at different levels** and even have **multiple instances** of the same service if needed.

1. **Platform Injector** — shared across all Angular apps on the page (rare case)
2. **Root Injector** — created at app bootstrap; holds `providedIn: 'root'` services (singletons for the whole app)
3. **Module Injectors** — for lazy-loaded modules, each gets its own injector
4. **Component Injectors** — every component can declare its own `providers: []` — creates a **new instance** scoped to that component and its children

- Component asks for a service → Angular checks the **component's own injector** first
- Not found? → moves up to the **parent component**
- Keeps bubbling up → finally reaches **root injector**
- If still not found → throws `NullInjectorError`

```
// Root-level singleton — ONE instance for entire app
@Injectable({ providedIn: 'root' })
export class AuthService { }

// Component-level — NEW instance per component tree
@Component({
  selector: 'app-wizard',
  providers: [WizardStateService]  // fresh instance here
})
export class WizardComponent { }
```

**Real Example:** In my project, `AuthService` is `providedIn: 'root'` — one shared instance across the whole app. But `FormWizardService` is provided at the **component level** — each wizard instance gets its own isolated state, so opening two wizards in different tabs doesn't cause them to share data accidentally.

- **Singleton services** for cross-cutting concerns (auth, logging, API)
- **Scoped services** for per-feature or per-component state
- **Mockable for testing** — override a provider at the test bed level
- **Tree-shakable** — `providedIn: 'root'` services are removed from the bundle if unused

- *What happens if two injectors provide the same service?* — The **closest one wins**. A component-level provider overrides the root provider for that component and its children.

- *What is `@Self()` and `@SkipSelf()`?* — Decorators that control DI lookup. `@Self()` forces Angular to only look in the component's own injector. `@SkipSelf()` skips it and starts from the parent.

---

# TOPIC 6: ROUTING

## Q1: What is Routing in Angular? (IMPORTANT)

**A:** Routing in Angular enables **seamless navigation between different views/components** without triggering a full page reload — this is what makes it a true **Single Page Application**. We define a mapping between **URL paths and components**, and Angular's `RouterModule` handles the rest.

```
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'users', component: UserListComponent },
  { path: 'users/:id', component: UserDetailComponent },
  { path: '**', component: NotFoundComponent }
];
```

- `<router-outlet>` — the placeholder in HTML where the routed component gets rendered
- `routerLink` — used for declarative navigation in templates
- `Router.navigate()` — used for programmatic navigation in TypeScript
- `**` **(wildcard)** — catches all unmatched URLs and shows a custom 404 page

**Real Example:** In my project, I set up routing so `/dashboard` loads the DashboardComponent, `/users/:id` loads user details with the ID from the URL, and `**` shows a custom "Page Not Found" component. The navigation feels **instant** because Angular only swaps the component inside `<router-outlet>`.

Routing is what transforms a regular Angular app into a .

- *What is `pathMatch: 'full'`?* — It means the **entire URL path** must match exactly, not just the prefix. Used mainly with empty-path redirects to avoid catching all routes.

## Q2: What is Lazy Loading? How do you implement it? (IMPORTANT)

**A:** Lazy loading is a **performance optimization technique** where feature modules are **loaded on demand** — only when the user actually navigates to that route — instead of being bundled into the initial app load. This **significantly reduces the initial bundle size** and makes the first page load much faster.

```
const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module')
      .then(m => m.AdminModule)
  }
];
```

```
{
  path: 'admin',
  loadComponent: () => import('./admin/admin.component')
    .then(c => c.AdminComponent)
}
```

**Real Example:** In my project, the Admin module had charts, reports, and user management — heavy code that only admins needed. After lazy loading it, the **main app bundle reduced by around 30%**, and regular users never download the admin code at all. This was one of the biggest performance wins we achieved.

Lazy loading is a for any production Angular application.

- *How do you verify lazy loading is working?* — Open the **Network tab** in browser DevTools. When you navigate to the lazy-loaded route, you'll see a **separate chunk file** being downloaded — that confirms it's working.
- *What is preloading strategy?* — After the initial load completes, Angular can **preload lazy modules in the background** using `PreloadAllModules` strategy — so they're ready instantly when the user navigates.

## Q3: What are Route Guards? (IMPORTANT)

**A:** Route Guards are **security checkpoints** that control whether a user can **navigate to, leave, or load** a particular route. They act as gatekeepers — returning `true` to allow access or `false` (or a

redirect `UrlTree`) to deny it.

| Guard | Purpose |
|---|---|
| **CanActivate** | Can the user **access** this route? |
| **CanDeactivate** | Can the user **leave** this route? (unsaved changes warning) |
| **CanActivateChild** | Can the user access **child routes**? |
| **Resolve** | Fetch data **before** the route loads |
| **CanLoad** | Should the lazy module **even be downloaded**? |

```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  if (authService.isLoggedIn()) {
    return true;
  }
  return inject(Router).createUrlTree(['/login']);
};
```

```
{ path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] }
```

**Real Example:** In my project, I use `CanActivate` on every protected route to verify the user's **JWT token is valid** before allowing access. If the token is expired, the guard redirects to the login page. I also use `CanDeactivate` on form pages to **warn users about unsaved changes** before they navigate away.

Route Guards are in any real-world app.

- *What is the difference between CanActivate and CanLoad?* — `CanActivate` prevents **accessing** the route but the lazy module still gets downloaded. `CanLoad` prevents the **entire module from even being downloaded** — better for security since unauthorized users can't even see the code.

---

## Q4: How do you pass data between routes?

**A:** Angular provides **3 distinct ways** to pass data during navigation, each suited for different scenarios:

```
// Route config
{ path: 'user/:id', component: UserComponent }

// Navigate
this.router.navigate(['/user', 123]);

// Read in target component
this.route.params.subscribe(params => {
  this.userId = params['id'];
});
```

```
// Navigate
this.router.navigate(['/users'], { queryParams: { page: 2, sort: 'name' } });
// URL: /users?page=2&sort=name

// Read
this.route.queryParams.subscribe(params => {
  this.page = params['page'];
});
```

```
this.router.navigate(['/user'], { state: { userData: this.user } });

// Read in target component
this.user = history.state.userData;
```

**Real Example:** In my project, I use **route params** for user detail pages ( `/user/42` ), **query params** for table filters and pagination ( `/users?page=2&status=active` ), and **router state** for passing the complete user object to an edit page without exposing it in the URL.

— params for required URL data, query params for optional filters, state for hidden complex data.

- *What is the difference between* `params` *and* `queryParams` *?* — `params` are **part of the URL path** ( `/user/123` ) and are required. `queryParams` come **after the** `?` ( `/users?page=2` ) and are optional. Params define the route, queryParams refine it.

---

## Q5: What is a Resolver?

**A:** A Resolver is a special route guard that **pre-fetches data before the route activates**. The target component only loads **after the data is completely ready** — so the user never sees an empty screen or a flash of loading state.

---

```typescript
export const userResolver: ResolveFn<User> = (route) => {
  return inject(UserService).getUser(route.params['id']);
};

// Route config
{ path: 'user/:id', component: UserComponent, resolve: { user: userResolver } }

// Access in component — data is already available!
constructor(private route: ActivatedRoute) {
  this.user = this.route.snapshot.data['user'];
}
```

In my project, I used a resolver for the user profile page — when you click on a user, the navigation waits until the API returns the user data, then renders the complete profile. No loading spinner needed.

Resolvers guarantee that **components always receive pre-loaded data**, creating a **smoother user experience**.

- *Downside of resolvers?* — Navigation **feels slower** because the user waits on a blank screen while data fetches. In many cases, it's actually better to load the component immediately and show a **skeleton loader** while the API call completes — feels more responsive.

---

# TOPIC 7: FORMS (TEMPLATE-DRIVEN + REACTIVE)

### Q1: What is the difference between Template-driven and Reactive Forms?

**(IMPORTANT)**

**A:** Angular provides **two approaches** for building forms, each designed for different complexity levels:

| Feature | Template-Driven | Reactive |
|---|---|---|
| **Defined in** | HTML (template) | TypeScript (component) |
| **Binding** | `[(ngModel)]` | `FormControl`, `FormGroup` |
| **Validation** | HTML attributes (`required`) | `Validators.required` in TS |
| **Module needed** | `FormsModule` | `ReactiveFormsModule` |
| **Best for** | Simple forms (login, contact) | Complex forms (registration, dynamic) |
| **Testing** | Harder (DOM-dependent) | Easier (pure TypeScript) |
| **Data flow** | Two-way binding (implicit) | Unidirectional (explicit control) |

**My approach:** For simple forms like login or contact, I use **Template-driven** because it's quicker to set up. For anything complex — **dynamic fields, conditional validation, multi-step wizards, FormArrays** — I always go with **Reactive Forms** because I get full programmatic control.

**Real Example:** In my project, the login page uses template-driven forms (just email + password). But the user registration form with dynamic address fields, conditional validation, and password matching uses **Reactive Forms** — much cleaner to manage in TypeScript.

Reactive Forms give you for complex scenarios.

- *Can you use both in the same project?* — Yes, absolutely. But not in the **same form**. Import both `FormsModule` and `ReactiveFormsModule` in your module.

---

## Q2: How do you create a Reactive Form? (IMPORTANT)

**A:** I use Angular's `FormBuilder` service to create a `FormGroup` containing `FormControl` fields with `Validators`. This gives me **full control over form structure, validation, and data flow** — all in TypeScript.

```ts
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

export class LoginComponent implements OnInit {
  loginForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit() {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]],
    });
  }

  onSubmit() {
    if (this.loginForm.valid) {
      console.log(this.loginForm.value);
    }
  }
}
```

```html
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <input formControlName="email" placeholder="Email">
  <div *ngIf="loginForm.get('email')?.errors?.['required']">
    Email is required
  </div>

  <input formControlName="password" type="password">
  <button type="submit" [disabled]="loginForm.invalid">Login</button>
</form>
```

**Real Example:** In my project, every form beyond basic login uses Reactive Forms. The registration form has **12+ fields with cross-field validation** (password match), conditional required fields, and dynamic sections — all managed cleanly through `FormBuilder`.

Reactive Forms are the .

- *How do you create a custom validator?* — Write a function that returns `null` (valid) or an error object (invalid):

```ts
function noSpaces(control: AbstractControl) {
  return control.value?.includes(' ') ? { noSpaces: true } : null;
}
```

## Q3: What is FormGroup, FormControl, and FormArray?

**A:** These are the **three core building blocks** of Reactive Forms, each serving a different level of form structure:

- **FormControl** — represents a **single input field** (like email, name, or a checkbox). It tracks the value, validation state, and user interactions.
- **FormGroup** — a **collection of FormControls** grouped together (typically the entire form). Validation can also be applied at the group level.
- **FormArray** — a **dynamic array** of FormControls or FormGroups. You can **add or remove fields at runtime** — perfect for dynamic forms.

```
// FormArray example — adding multiple phone numbers dynamically
this.form = this.fb.group({
  name: [''],
  phones: this.fb.array([this.fb.control('')])
});

addPhone() {
  (this.form.get('phones') as FormArray).push(this.fb.control(''));
}
```

**Real Example:** I used `FormArray` in a form where users could add **multiple shipping addresses** — clicking "Add Address" dynamically creates a new address FormGroup with street, city, and pincode fields. The user can add or remove addresses freely.

FormArray is the that adapt to user needs at runtime.

## Q4: How do you handle form validation? (IMPORTANT)

**A:** Angular provides a robust validation system with both **built-in validators** and support for **custom validators**. The key is combining proper validation rules with **user-friendly error messages** that only appear at the right time.

**Built-in validators:** `Validators.required`, `Validators.email`, `Validators.minLength()`, `Validators.maxLength()`, `Validators.pattern()`

```
<input formControlName="email">
<div *ngIf="loginForm.get('email')?.touched && loginForm.get('email')?.invalid">
  <span *ngIf="loginForm.get('email')?.errors?.['required']">Email is required</span>
  <span *ngIf="loginForm.get('email')?.errors?.['email']">Invalid email format</span>
</div>
```

- `touched` — user has visited and left the field
- `dirty` — user has actually changed the value
- `valid` / `invalid` — current validation status
- `errors` — object containing specific error details

**Real Example:** In my project, I show validation errors only when the field is **touched AND invalid** — this prevents showing errors before the user even interacts with the form. For the submit button, I use `[disabled]="form.invalid"` to prevent submission of incomplete forms.

Proper validation ensures .

- *What is cross-field validation?* — When validation depends on **multiple fields together** (e.g., password and confirm password must match). It's implemented at the **FormGroup level** as a group validator, not on individual FormControls.

---

## Q5: What is ngModel and two-way binding?

**A:** `ngModel` enables **two-way data binding** between a form input and a component property. Changes in the input **instantly update the variable**, and changes to the variable **instantly update the input** — they stay perfectly in sync automatically.

```
// component.ts
username: string = '';
```

```
<!-- component.html -->
<input [(ngModel)]="username" placeholder="Enter name">
<p>Hello, {{ username }}!</p>
```

The `[()]` syntax is called **"banana in a box"** — it's a combination of **property binding** `[]` (data flows in) and **event binding** `()` (data flows out). You must import `FormsModule` to use it.

**Real Example:** In my project's quick-search feature, I use `[(ngModel)]` on the search input — as the user types, the `searchTerm` variable updates instantly, and I use it to filter a local list in real-time

without any button click.

Two-way binding is .

---

## Q6: How do Angular Forms work internally?

**A:** Internally, all Angular forms — whether template-driven or reactive — are built on the same **core abstractions** from `@angular/forms`. Understanding this helps you debug form issues and write custom form controls.

| Class | Role |
|---|---|
| **AbstractControl** | Base class — every form control, group, and array extends it |
| **FormControl** | Represents a single input field — tracks value, validity, touched/dirty state |
| **FormGroup** | A collection of named controls — the form itself |
| **FormArray** | A dynamic array of controls — for repeating fields |
| **ControlValueAccessor (CVA)** | The **bridge** between DOM elements and FormControl — how Angular reads/writes values |
| **NgForm / FormGroupDirective** | Directives that connect templates to the model |

1. You create a `FormControl` (either via `FormBuilder` in TS, or via `ngModel` in the template).
2. The **ControlValueAccessor** connects the DOM `<input>` to the `FormControl` — calling `writeValue()` when the model changes and notifying via `onChange()` when the user types.
3. Every keystroke → CVA emits the new value → `FormControl` updates its `value` → runs all attached **validators** → updates `status` (VALID/INVALID) → emits `valueChanges` and `statusChanges` observables.
4. Angular's **change detection** picks up the new state and re-renders the template (error messages, disabled states, etc.).

```
// Under the hood — every form field is an AbstractControl
form = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email])
});

// These Observables let you react to any change
this.form.valueChanges.subscribe(val ⇒ console.log(val));
this.form.statusChanges.subscribe(status ⇒ console.log(status));
```

**Real Example:** In my project, I built a **custom date-range picker** as a reusable form control by implementing `ControlValueAccessor`. Because I followed the same contract Angular's built-in inputs use, my custom component works seamlessly with Reactive Forms, validators, and `[(ngModel)]` — no extra wiring needed.

- You can build **custom form controls** that integrate perfectly with Angular's validation system
- You can tap into `valueChanges` for powerful reactive behavior (live search, auto-save, cross-field validation)
- Understanding this pipeline makes debugging form issues **much faster**

- *What are the 4 methods of ControlValueAccessor?* — `writeValue()`, `registerOnChange()`, `registerOnTouched()`, and optional `setDisabledState()`.
- *Which approach uses these classes more directly — template-driven or reactive?* — **Reactive Forms** — you instantiate `FormGroup` / `FormControl` explicitly. Template-driven forms create them **implicitly** behind the scenes via `ngModel`.

---

# TOPIC 8: HTTP & API HANDLING

### Q1: How do you make HTTP requests in Angular? (IMPORTANT)

**A:** Angular provides the `HttpClient` service from `@angular/common/http` for all HTTP communication. It returns **Observables** by default, which gives us powerful capabilities like **cancellation, retry, and operator chaining** through RxJS.

```typescript
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {
  private apiUrl = 'https://api.example.com';

  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(`${this.apiUrl}/users`);
  }

  createUser(user: User): Observable<User> {
    return this.http.post<User>(`${this.apiUrl}/users`, user);
  }

  updateUser(id: number, user: User): Observable<User> {
    return this.http.put<User>(`${this.apiUrl}/users/${id}`, user);
  }

  deleteUser(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/users/${id}`);
  }
}
```

```typescript
// In component
this.userService.getUsers().subscribe({
  next: (users) => this.users = users,
  error: (err) => console.error('Failed:', err)
});
```

**Real Example:** In my project, I follow a clean pattern — all HTTP calls live in **dedicated services**, components just call service methods and subscribe. This makes the code **testable** (I can mock the service) and **reusable** (multiple components use the same service).

HttpClient with Observables is the to handle API communication in Angular.

- *Why does HttpClient return Observable instead of Promise?* — Observables are **cancelable** (unsubscribe stops the request), support **retry logic**, can emit **multiple values**, and integrate seamlessly with **RxJS operators** for data transformation.

## Q2: What are HTTP Interceptors? (IMPORTANT)

**A:** Interceptors are like **middleware for every HTTP request and response** in the app. They sit between the application and the server, allowing us to **modify requests or handle responses globally** — without changing individual service calls. Common uses include **attaching auth tokens, logging, global error handling, and showing loading spinners**.

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');

  if (token) {
    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
    return next(authReq);
  }

  return next(req);
};
```

```
provideHttpClient(withInterceptors([authInterceptor]))
```

**Real Example:** In my project, I had 2 interceptors working as a **pipeline**:

1. **Auth interceptor** — automatically attaches the JWT token to every outgoing request
2. **Error interceptor** — catches 401 responses globally and redirects to the login page

Without interceptors, I would have to add token logic in — interceptors saved hundreds of lines of duplicate code.

Interceptors are in any production Angular app.

- *Can you have multiple interceptors?* — Yes, they execute in the **exact order they are registered** — like a pipeline. The request passes through each interceptor sequentially.
- *Why do we clone the request?* — HTTP requests in Angular are **immutable objects**. We can't modify the original, so we `clone()` it with the changes we need.

### Q3: How do you handle errors in HTTP calls? (IMPORTANT)

**A:** I handle errors at **multiple levels** for a robust error-handling strategy — service level for specific handling, and interceptor level for global catch-all behavior.

```
getUsers(): Observable<User[]> {
  return this.http.get<User[]>('/api/users').pipe(
    retry(2), // retry 2 times before failing
    catchError(error => {
      console.error('API Error:', error);
      this.notificationService.showError('Failed to load users');
      return of([]); // return empty array as fallback
    })
  );
}
```

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401) {
        inject(Router).navigate(['/login']);
      } else if (error.status === 500) {
        inject(NotificationService).showError('Server error');
      }
      return throwError(() => error);
    })
  );
};
```

**Real Example:** In my project, the error interceptor handles **401 (redirect to login)**, **403 (access denied toast)**, **500 (server error message)**, and **status 0 (network disconnected warning)**. Individual services handle specific cases like showing "No users found" when a search returns empty.

A ensures no error goes unnoticed while keeping the user informed at every step.

- *What is the difference between* `of()` *and* `throwError()`*?* — `of()` returns a **fallback value** and the stream continues normally. `throwError()` **re-throws** the error so the subscriber's `error` callback catches it.

## Q4: What is the difference between REST API and SOAP API?

**A:** REST and SOAP are two different **architectural approaches for building APIs**. In modern web development, REST has become the dominant choice due to its **simplicity and lightweight nature**.

| Feature | REST API | SOAP API |
|---|---|---|
| **Protocol** | HTTP only | HTTP, SMTP, TCP |
| **Data format** | JSON (mainly), XML | XML only (strict) |
| **Speed** | Faster, lightweight | Slower, heavy overhead |
| **Ease of use** | Simple, flexible | Complex, strict standards |
| **Stateless** | Yes (each request is independent) | Can maintain state |
| **Use case** | Web apps, mobile apps, SPAs | Banking, enterprise, legacy systems |

**Real Example:** In all my Angular projects, I've worked exclusively with **REST APIs** because they return **JSON** — which maps directly to TypeScript interfaces. Parsing and type-checking are effortless. SOAP with its XML envelope and strict schemas would be overkill for typical web applications.

REST APIs are the .

## Q5: How do you implement caching in Angular?

**A:** Caching avoids **redundant API calls**, reduces server load, and makes the UI feel instant. There are several strategies depending on the use case — from simple in-memory caching to interceptor-level caching with TTL.

(simplest, for reference data)

```ts
@Injectable({ providedIn: 'root' })
export class CountryService {
  private countries$?: Observable<Country[]>;

  getCountries(): Observable<Country[]> {
    if (!this.countries$) {
      this.countries$ = this.http.get<Country[]>('/api/countries').pipe(
        shareReplay(1)  // cache the result, replay to any subscriber
      );
    }
    return this.countries$;
  }
}
```

First call hits the API. Every subsequent subscriber — no extra HTTP call.

```ts
@Injectable({ providedIn: 'root' })
export class CacheService {
  private cache = new Map<string, { data: any; expiry: number }>();

  get<T>(key: string, fetch: () => Observable<T>, ttl = 60000): Observable<T> {
    const cached = this.cache.get(key);
    if (cached && cached.expiry > Date.now()) {
      return of(cached.data);
    }
    return fetch().pipe(
      tap(data => this.cache.set(key, { data, expiry: Date.now() + ttl }))
    );
  }
}
```

(app-wide, transparent)

An interceptor caches GET responses by URL — subsequent identical requests skip the network entirely. Ideal for dashboards where the same lookups happen across many components.

**Strategy 4: Browser storage** — `localStorage` or `sessionStorage` for data that should survive page reloads (user preferences, feature flags).

**Real Example:** In my project, a `CountryService` and `CurrencyService` returned static lookup lists that changed once a week. Using `shareReplay(1)`, we went from **30+ API calls per session** to just **one** — a major load reduction on the backend and noticeably faster form rendering.

- Only cache **safe** data (GET requests, reference data) — never cache sensitive user-specific info on shared storage
- Always define a **clear invalidation strategy** — how the cache gets refreshed after edits

- Expose a `clearCache()` method for logout flows
- *Difference between* `shareReplay` *and* `publishReplay().refCount()` *?* — `shareReplay` is the modern, preferred operator — simpler API and works as expected in most cases.
- *How do you invalidate the cache after a POST/PUT?* — Reset the cached observable (`this.countries$ = undefined`) or call `cacheService.clear('countries')`.

---

## Q6: How do you handle multiple API calls?

**A:** Handling multiple API calls is a very common interview scenario. RxJS gives us several operators — the right one depends on whether the calls are **parallel, sequential, or dependent** on each other.

(wait for all, combine results)

```
// Load dashboard data — 3 APIs in parallel
forkJoin({
  users: this.http.get<User[]>('/api/users'),
  orders: this.http.get<Order[]>('/api/orders'),
  stats: this.http.get<Stats>('/api/stats')
}).subscribe(({ users, orders, stats }) => {
  this.initDashboard(users, orders, stats);
});
```

Fires all 3 calls simultaneously. Emits , when all complete. If any fails, the whole thing fails.

```
// First get user, then get that user's orders
this.http.get<User>('/api/user/me').pipe(
  switchMap(user => this.http.get<Order[]>(`/api/orders?userId=${user.id}`))
).subscribe(orders => this.orders = orders);
```

When you want to show partial results as they stream in (e.g., notifications from multiple sources).

```
// Dashboard that re-renders whenever filter OR search changes
combineLatest([this.filter$, this.search$]).pipe(
  debounceTime(200),
  switchMap(([filter, search]) => this.api.query(filter, search))
).subscribe(results => this.results = results);
```

When each request must finish before the next starts (e.g., saving 10 items in order).

**Real Example:** In my project's analytics dashboard, `forkJoin` loads revenue, user stats, and recent orders in parallel — all 3 APIs fire together, and the UI shows a single spinner until all complete. This was 3× faster than the previous sequential implementation that loaded them one after another.

- **Independent, wait for all** → `forkJoin`
- **Each depends on the previous** → `switchMap` / `concatMap`
- **Need latest from multiple streams** → `combineLatest`
- **Fire and merge as they arrive** → `mergeMap` / `merge`

- *What if one API fails in `forkJoin`?* — The whole observable fails. To handle failures gracefully, wrap each inner call with `catchError(() ⇒ of(defaultValue))` so it "succeeds" with a fallback.
- *Difference between `forkJoin` and `combineLatest`?* — `forkJoin` emits **once, at the end**. `combineLatest` emits **every time any source emits** — continuous updates.

---

# TOPIC 9: RXJS BASICS

### Q1: What is RxJS and why is it used in Angular? (IMPORTANT)

**A:** RxJS (Reactive Extensions for JavaScript) is a **library for reactive programming** using **Observables**. Angular has RxJS deeply integrated — **HttpClient, Router events, Form value changes, ViewChild queries** — all return Observables. Understanding RxJS is essentially **mandatory for Angular development**.

- Can emit **multiple values** over time (not just one)
- **Cancelable** — `unsubscribe()` stops execution
- Has 100+ powerful **operators** for transforming, filtering, and combining data streams
- Perfect for **real-time scenarios** — WebSockets, live search, user input streams

**Real Example:** I use RxJS heavily for live search — `debounceTime(300)` waits for the user to stop typing, `distinctUntilChanged()` skips if the search term hasn't changed, and `switchMap` cancels the previous API call and fires a new one. This entire pipeline would take 30+ lines of imperative code — RxJS does it in **5 lines**.

RxJS is the and mastering it is what separates junior from mid-level developers.

## Q2: What is the difference between Observable and Promise? (IMPORTANT)

**A:** Both handle asynchronous operations, but Observables are **significantly more powerful** and flexible than Promises:

| Feature | Observable | Promise |
|---|---|---|
| **Values** | **Multiple** values over time | Only **one** value |
| **Lazy** | Yes — nothing happens until `subscribe()` | No — executes immediately on creation |
| **Cancelable** | Yes — `unsubscribe()` cancels | No — once started, can't stop |
| **Operators** | 100+ operators ( `map` , `filter` , `switchMap` ) | Only `.then()` and `.catch()` |
| **Built into Angular** | Yes — core of HttpClient, Forms, Router | Supported but less common |

**Simple way to explain:** A Promise is like **ordering a pizza** — you get one delivery and it's done. An Observable is like a **Netflix subscription** — you keep getting new content over time, and you can **cancel anytime** you want.

**Real Example:** In my project, I use Observables for API calls (HTTP), form value tracking, and real-time notifications. The ability to **chain operators** and **cancel ongoing requests** makes them far superior to Promises for Angular apps.

Observables are the .

## Q3: What are commonly used RxJS operators? (IMPORTANT)

**A:** RxJS operators are **functions that transform, filter, or combine Observable streams**. Knowing the right operator for the right situation is crucial for writing efficient Angular code.

| Operator | What it does | Real Use Case |
|---|---|---|
| `map` | Transform emitted values | Extract specific fields from API response |
| `filter` | Pass only values meeting a condition | Ignore empty search terms |
| `switchMap` | Cancel previous, switch to new Observable | **Search-as-you-type** (most asked!) |
| `mergeMap` | Handle multiple inner Observables in parallel | Parallel file uploads |
| `concatMap` | Handle Observables in strict sequence | Sequential form submissions |
| `debounceTime` | Wait N ms before emitting | Search input delay (avoid spam calls) |
| `distinctUntilChanged` | Skip consecutive duplicate values | Prevent duplicate API calls |
| `catchError` | Handle errors gracefully | Show fallback data on API failure |
| `takeUntil` | Auto-unsubscribe when signal fires | **Memory leak prevention** |
| `tap` | Side effects without changing data | Logging, debugging |
| `combineLatest` | Combine latest values from multiple streams | Dashboard needing data from 3 APIs |

**Most asked in interviews:** `switchMap`, `debounceTime`, `catchError`, and `takeUntil` — know these with real examples and you'll impress the interviewer.

---

### Q4: Explain switchMap with a real example. (IMPORTANT)

**A:** `switchMap` is the **most important RxJS operator for interviews**. It does one powerful thing — when a new value comes in, it **cancels the previous inner Observable** and switches to a new one. This makes it perfect for **search-as-you-type** scenarios.

Here's what happens:

- User types "An" → API call 1 starts
- User types "Ang" → API call 1 is **immediately canceled**, API call 2 starts
- User types "Angular" → API call 2 is **canceled**, API call 3 starts

Only the reaches the subscriber. No race conditions, no stale data.

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),          // wait 300ms after user stops typing
  distinctUntilChanged(),     // skip if same search term
  switchMap(term =>           // cancel previous, make new API call
    this.http.get(`/api/search?q=${term}`)
  )
).subscribe(results => {
  this.searchResults = results;
});
```

**Real Example:** In my project's product search, before using `switchMap`, users sometimes saw **stale results** from an older, slower API call overwriting newer results. After implementing this pattern, the search became **perfectly reliable** — always showing results for the latest query.

`switchMap` is the **go-to operator for any scenario where only the latest request matters**.

- *When would you use mergeMap instead?* — When I **don't want to cancel** previous operations. Like saving multiple items to the server — each save should complete independently, regardless of new saves being triggered.

---

## Q5: What is Subject and BehaviorSubject? (IMPORTANT)

**A:** Both are **special types of Observables** that can act as both **producer and consumer** — meaning they can emit values AND be subscribed to. The key difference is how they handle **late subscribers**.

| Feature | Subject | BehaviorSubject |
|---|---|---|
| **Initial value** | No initial value | Requires an initial value |
| **Late subscribers** | Get **nothing** (miss all past values) | Get the **last emitted value** immediately |
| **Use case** | One-time events (button clicks, actions) | Sharing **current state** (logged-in user, settings) |

```
// BehaviorSubject — always holds a current value
private userSubject = new BehaviorSubject<User | null>(null);
user$ = this.userSubject.asObservable();

login(user: User) {
  this.userSubject.next(user);
}

// Any component subscribing — even 10 seconds later — gets the current user
```

**Real Example:** I use `BehaviorSubject` in my `AuthService` to hold the **currently logged-in user**. The header component, sidebar, and profile page all subscribe to it. Even if the header component loads **after** login, it immediately gets the user data — no timing issues.

BehaviorSubject is the in Angular.

- *What about ReplaySubject?* — `ReplaySubject(N)` replays the **last N values** to new subscribers. Useful when you need a short history — like the last 3 notifications — not just the latest one.

---

## Q6: When should you unsubscribe from Observables and why?

**A:** Unsubscribing prevents **memory leaks** — when a component is destroyed but its subscriptions keep running, they hold references in memory, may execute callbacks on a destroyed view, and can cause duplicate work or unexpected behavior.

| Source | Why |
|---|---|
| **BehaviorSubject / Subject streams** in services | Long-lived; don't auto-complete |
| **interval / timer** (RxJS or `setInterval`) | Run forever until stopped |
| **WebSocket / SSE / EventSource** | Persistent connections |
| **Form** `valueChanges` / `statusChanges` | Live as long as the form lives |
| **Router** `events` | Continuous stream throughout the app |
| **Manual** `fromEvent` **DOM listeners** | Stay attached to the DOM |

- **HttpClient calls** — they auto-complete after one emission
- `ActivatedRoute.params` / `queryParams` — Angular cleans them up
- **Observables consumed via** `async` **pipe** — pipe auto-unsubscribes on destroy
- **Anything wrapped with** `take(1)` / `first()` / `takeUntilDestroyed()` — completes itself

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => this.handle(data));
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

constructor() {
  this.service.data$
    .pipe(takeUntilDestroyed())  // auto-cleans on destroy
    .subscribe(...);
}
```

**Real Example:** In my project, a notification component had a `BehaviorSubject` subscription that was never unsubscribed. Every time the user navigated away and back, **a new subscription stacked on top of the old ones** — after 50 navigations, every incoming notification triggered the callback 50 times. Adding `takeUntil` immediately fixed it.

- **Memory leaks** — orphan subscriptions keep references alive forever
- **Duplicate side effects** — multiple stale subscribers reacting to one emission
- **Errors after destroy** — callbacks updating component state that no longer exists
- *Why does HTTP not need unsubscribing?* — Because `HttpClient` observables **complete after one emission**. Completed observables auto-clean their resources.
- *What's the safest single approach?* — Combine `async` **pipe** for templates + `takeUntilDestroyed()` for any imperative subscriptions. Together they handle 95% of real-world cases.

---

## Q7: How do you optimize API calls with RxJS?

**A:** RxJS provides several powerful operators specifically designed to **reduce, debounce, cache, and combine** API calls — making apps faster, cheaper, and more reliable.

```
this.search.valueChanges.pipe(
  debounceTime(300)  // ignore rapid input
).subscribe(...);
```

Reduces 10 keystrokes → 1 API call.

```
.pipe(distinctUntilChanged())  // skip if same as previous emission
```

Prevents duplicate calls when the user retypes the same query.

```
.pipe(switchMap(term => this.api.search(term)))
```

When a new request fires, the old one is — no stale results overwriting newer ones.

```
this.config$ = this.http.get('/api/config').pipe(shareReplay(1));
```

First subscriber triggers the call. Every other subscriber gets the cached value — .

```
forkJoin([this.api.users(), this.api.orders(), this.api.stats()])
  .subscribe(([users, orders, stats]) => ...);
```

3 calls in parallel instead of sequential = ~3× faster.

```
this.http.get('/api/data').pipe(retry({ count: 3, delay: 1000 }))
```

```
.pipe(catchError(() => of([])))  // return empty list instead of failing
```

**Real Example:** A live-search feature in my project initially fired ~15 API calls per search session. After applying `debounceTime(300) + distinctUntilChanged() + switchMap`, it dropped to **1–2 calls per search** — an 85%+ reduction in backend load and a much smoother UX.

```
this.results$ = this.search.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  filter(q => q.length >= 2),
  switchMap(q => this.api.search(q).pipe(
    catchError(() => of([]))
  )),
  shareReplay(1)
);
```

- *What's the difference between* `retry` *and* `repeat`*?* — `retry` resubscribes when the source **errors**. `repeat` resubscribes when the source **completes successfully** — useful for polling.
- *How do you implement polling with RxJS?* — `interval(5000).pipe(switchMap(() => this.api.getStatus()))` — fires every 5 seconds, cancels in-flight calls if a new tick comes.

---

# TOPIC 10: LIFECYCLE HOOKS

### Q1: What are Lifecycle Hooks? Name the important ones. (IMPORTANT)

**A:** Lifecycle hooks are **predefined methods** that Angular calls at **specific stages of a component's life** — from creation to rendering to destruction. They give us the ability to **run custom logic at exactly the right moment**.

| Hook | When it runs | Common use |
|---|---|---|
| `constructor` | When the class is instantiated | Inject dependencies only |
| `ngOnChanges` | Before `ngOnInit` & whenever `@Input()` changes | React to parent data changes |
| `ngOnInit` | After first `ngOnChanges` | **API calls, component initialization** |
| `ngAfterViewInit` | After the view and child views are rendered | Access DOM via `@ViewChild` |
| `ngOnDestroy` | Just before the component is removed | **Unsubscribe, cleanup resources** |

- `ngOnInit` — for initial API calls, setting up subscriptions, and initialization logic

- `ngOnDestroy` — for unsubscribing from Observables and cleaning up timers to prevent **memory leaks**

**Real Example:** In my project, every component that makes API calls follows this pattern — fetch data in `ngOnInit`, and clean up all subscriptions in `ngOnDestroy` using the `takeUntil` pattern.

Lifecycle hooks give you at every stage of its existence.

---

## Q2: What is the difference between constructor and ngOnInit? (IMPORTANT)

**A:** This is one of the **most frequently asked Angular interview questions**. The constructor is a **TypeScript/JavaScript feature** — it runs when the class is instantiated. `ngOnInit` is an **Angular lifecycle hook** — it runs after Angular has fully set up the component, including its `@Input()` bindings.

| Feature | constructor | ngOnInit |
|---|---|---|
| **What is it** | TypeScript class feature | Angular lifecycle hook |
| **When it runs** | When class instance is created | After `@Input()` bindings are ready |
| **Use for** | **Only dependency injection** | **API calls, initialization logic** |
| `@Input()` **available?** | No — not yet set | Yes — fully available |

```
export class UserComponent implements OnInit {
  @Input() userId!: number; // NOT available in constructor

  constructor(private userService: UserService) {} // only inject

  ngOnInit() {
    // userId is available here — safe to use
    this.userService.getUser(this.userId).subscribe(user => {
      this.user = user;
    });
  }
}
```

Constructor = inject dependencies. ngOnInit = do the actual work. This keeps the code predictable and avoids timing issues.

- *Why not just do everything in the constructor?* — Because `@Input()` values are **not set yet** in the constructor. Angular sets them **after** construction but **before** `ngOnInit`. If you try to use `@Input()` data in the constructor, you'll get `undefined`.

### Q3: What is ngOnDestroy? Why is it important? (IMPORTANT)

**A:** `ngOnDestroy` is the **cleanup hook** — it runs **just before Angular removes the component from the DOM**. Its primary purpose is to **free up resources** — especially **unsubscribing from Observables** to prevent **memory leaks** that can degrade app performance over time.

```
export class UserComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    this.userService.getUsers()
      .pipe(takeUntil(this.destroy$))
      .subscribe(users => this.users = users);

    this.sharedService.data$
      .pipe(takeUntil(this.destroy$))
      .subscribe(data => this.data = data);
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

- Clear `setInterval` / `setTimeout` references
- Close WebSocket connections
- Remove manual DOM event listeners
- Disconnect from third-party libraries

**Real Example:** In my project, we once had a **memory leak issue** — a component with a WebSocket subscription was being created and destroyed on route changes, but the subscription kept running in the background. After adding proper cleanup in `ngOnDestroy`, the memory leak disappeared and app performance stabilized.

`ngOnDestroy` is **non-negotiable for production apps** — skipping it leads to memory leaks, phantom subscriptions, and unpredictable behavior.

- *What happens if you don't unsubscribe?* — The subscription keeps running **even after the component is destroyed** — it becomes a **zombie subscription** that causes memory leaks, duplicate API calls, and unexpected side effects.

## Q4: What is ngOnChanges? When does it fire?

**A:** `ngOnChanges` fires **every time an** `@Input()` **property changes** from the parent component. It provides a `SimpleChanges` object containing the **previous and current values**, plus whether it's the **first change**. This is very useful when you need to **react to input changes** and perform logic based on what changed.

```
ngOnChanges(changes: SimpleChanges) {
  if (changes['userId'] && !changes['userId'].firstChange) {
    // userId changed (not the initial load) — reload user data
    this.loadUser(changes['userId'].currentValue);
  }
}
```

**Important caveat:** `ngOnChanges` only detects changes in **value/reference**, not mutations. If you mutate an object or push to an array, Angular won't detect it. You need to create a **new reference** — `[...array]` or `{...obj}` — for `ngOnChanges` to fire.

**Real Example:** In my project, I have a `UserDetailComponent` that receives `userId` as `@Input()`. When the parent changes the selected user, `ngOnChanges` detects it and **reloads the user data** — without this, the component would show stale data.

`ngOnChanges` is essential for components that need to **react dynamically to parent data changes**.

## Q5: What is ngAfterViewInit?

**A:** `ngAfterViewInit` fires **after the component's view and all child views are fully initialized and rendered**. This is the earliest point where you can safely access **DOM elements** or **child components** via `@ViewChild`.

```
@ViewChild('chartCanvas') chartCanvas!: ElementRef;

ngAfterViewInit() {
  // Canvas element is now available in the DOM
  this.initChart(this.chartCanvas.nativeElement);
}
```

**Real Example:** In my project, I integrated a third-party charting library that needed a **reference to a canvas element**. I used `@ViewChild` to get the element and initialized the chart in `ngAfterViewInit`

— trying to do it in `ngOnInit` would fail because the DOM isn't ready yet.

**Important gotcha:** If you modify component data inside `ngAfterViewInit`, you may get the dreaded `ExpressionChangedAfterItHasBeenChecked` error. The fix is to use `setTimeout()` or `ChangeDetectorRef.detectChanges()` to trigger a new change detection cycle.

`ngAfterViewInit` is the **go-to hook for any DOM manipulation or third-party library initialization**.

---

# TOPIC 11: STATE MANAGEMENT (BASIC)

## Q1: What is State Management? Why do we need it? (IMPORTANT)

**A:** State is the **data your application holds at any given moment** — the logged-in user, shopping cart items, applied filters, notification count, etc. State management is the strategy we use to **store, update, and share** this data predictably across all components that need it.

- In small apps, **services with BehaviorSubject** work perfectly fine
- In larger apps, when **multiple components** read and write the **same data**, things get messy — updates happen from different places, data goes out of sync, and bugs become hard to trace
- A proper state management approach gives you a **single source of truth** and a **predictable, traceable pattern** for all data changes

**Real Example:** In my project's e-commerce dashboard, the cart count is shown in the header, the cart details in the sidebar, and the total in the checkout page — all different components needing the **same cart state**. Using a centralized service with BehaviorSubject, any update to the cart **automatically reflects everywhere**.

State management ensures across the entire application.

- *Do you always need NgRx?* — No! For most mid-size apps, **services with BehaviorSubject** are more than enough. NgRx adds significant boilerplate and complexity — use it only when the app has **highly complex shared state** with many components reading and writing simultaneously.

---

## Q2: How do you manage state using Services? (IMPORTANT)

**A:** I use a **service with BehaviorSubject** as a lightweight, centralized state store. The service holds the state privately, exposes it as an Observable for components to subscribe to, and provides **clean methods to update the state** — giving us a **mini Redux pattern** without any external library.

```typescript
@Injectable({ providedIn: 'root' })
export class CartService {
  private cartItems = new BehaviorSubject<CartItem[]>([]);
  cartItems$ = this.cartItems.asObservable();

  addItem(item: CartItem) {
    const current = this.cartItems.value;
    this.cartItems.next([...current, item]);
  }

  removeItem(id: number) {
    const updated = this.cartItems.value.filter(i => i.id !== id);
    this.cartItems.next(updated);
  }

  getTotal(): number {
    return this.cartItems.value.reduce((sum, i) => sum + i.price, 0);
  }
}
```

Any component can subscribe to `cartItems$` and receive **automatic real-time updates** whenever the cart changes. The state is always **immutable** — we create new arrays instead of mutating.

**Real Example:** In my project, this exact pattern powers the cart, user preferences, and notification state. It's **simple, testable, and performant** — no external dependencies needed.

Service-based state management is the for most Angular applications.

---

## Q3: What is NgRx? When would you use it?

**A:** NgRx is a **full-featured state management library** for Angular based on the **Redux pattern**. It provides a structured, predictable way to manage complex application state with strict rules about how state can change.

- **Store** — single source of truth holding all application state
- **Actions** — plain objects describing what happened ("Add Item", "Login Success")
- **Reducers** — pure functions that take current state + action and return new state
- **Selectors** — efficiently select and derive specific slices of state
- **Effects** — handle side effects like API calls, triggered by actions

- Large app with **complex shared state** across many features
- Multiple developers need a **strict, enforced pattern** for state changes
- You need **time-travel debugging** and **state change traceability**

**When NOT to use:** Small to medium apps — NgRx adds **significant boilerplate** (actions, reducers, effects for every feature). Service-based state management is simpler and sufficient for most cases.

**Real Example:** In my current projects, service-based state management handles everything well. But I understand the Redux pattern thoroughly and can adopt NgRx when the project demands it — especially in **large enterprise apps with 20+ developers** where enforced patterns prevent chaos.

NgRx is .

- *Have you used NgRx?* — "I understand the Redux pattern and NgRx concepts well. In my projects, service-based state management was sufficient. But I'm comfortable with the architecture and ready to use NgRx when the project requires it."

---

## Q4: What are Angular Signals? (Angular 16+)

**A:** Signals are a **new reactive primitive** introduced in Angular 16 that provide a **simpler, synchronous way to manage reactive state**. A signal wraps a value and automatically notifies Angular when that value changes — no need for `subscribe()`, `async` pipe, or manual change detection.

```
import { signal, computed } from '@angular/core';

// Create a signal
count = signal(0);

// Computed signal (auto-updates when dependencies change)
doubleCount = computed(() => this.count() * 2);

// Update signal
increment() {
  this.count.update(val => val + 1);
}
```

```
<p>Count: {{ count() }}</p>
<p>Double: {{ doubleCount() }}</p>
<button (click)="increment()">+1</button>
```

- **Much simpler** than RxJS for component-level state
- **Better performance** — Angular knows exactly which parts of the template changed
- No `subscribe` / `unsubscribe` / memory leak concerns
- Key step toward **zoneless Angular** (removing Zone.js dependency)

**Real Example:** In newer parts of my project, I've started using Signals for **local component state** — like toggle flags, counters, and form status. For HTTP calls and complex async operations, I still use RxJS Observables. They complement each other well.

Signals represent the — definitely worth learning and adopting.

---

# TOPIC 12: PERFORMANCE OPTIMIZATION

### Q1: How do you improve Angular app performance? (IMPORTANT)

**A:** Performance optimization is something I actively think about in every project. Here are the **top strategies I follow and have implemented**:

**1. Lazy Loading** — Load feature modules only when the user navigates to them. This alone **reduced our initial bundle by 30%** in my project.

**2. OnPush Change Detection** — Skip unnecessary re-checks. Components only update when `@Input()` reference changes, events fire, or you manually trigger it.

— Prevent Angular from re-rendering entire lists. Only the changed items update in the DOM.

— Pre-compile templates at build time. Faster startup, smaller bundle, compile-time error checking.

**5. Proper Unsubscription** — Use `takeUntil` pattern or `async` pipe to prevent memory leaks from zombie subscriptions.

**6. Avoid function calls in templates** — `{{ calculate() }}` runs on **every change detection cycle**. Use computed values, variables, or **pure pipes** instead.

**7. Pure Pipes over methods** — Pipes with `pure: true` only re-execute when input reference changes — highly efficient.

**8. Virtual Scrolling** — For large lists (1000+ items), render only visible items using `@angular/cdk` ScrollingModule.

**9. Bundle Analysis** — Use `webpack-bundle-analyzer` to identify and remove heavy, unused dependencies.

**10. Image Optimization** — Use Angular's `NgOptimizedImage` directive for lazy loading and responsive images.

**Real Example:** In my project, combining lazy loading + OnPush + virtual scrolling brought the dashboard load time from **4 seconds to under 1.5 seconds** — a dramatic improvement that users immediately noticed.

## Q2: What is OnPush Change Detection? How does it help? (IMPORTANT)

**A:** By default, Angular checks **every component in the entire tree** on every change detection cycle — even components whose data hasn't changed. **OnPush strategy** tells Angular to only re-check a component when one of these things happens:

- `@Input()` **reference** changes (not internal mutation)
- An **event** fires inside the component (click, keypress, etc.)
- You manually call `ChangeDetectorRef.markForCheck()`
- An **Observable bound with async pipe** emits a new value

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class UserListComponent {
  @Input() users!: User[];
}
```

**Critical rule with OnPush** — always use **immutable patterns**:

```
// WRONG — mutates array, OnPush won't detect it
this.users.push(newUser);

// CORRECT — creates new reference, OnPush detects it
this.users = [...this.users, newUser];
```

**Real Example:** In my project, after switching a heavy data table component (500+ rows) to OnPush, the change detection cycles **dropped by over 60%**. The table went from sluggish scrolling to **buttery smooth** — it was one of the biggest performance wins.

OnPush is a that every Angular developer should use for performance-critical components.

## Q3: How do you handle memory leaks in Angular? (IMPORTANT)

**A:** Memory leaks in Angular typically happen when **subscriptions keep running** after a component is destroyed — the subscription becomes a "zombie" that consumes memory and can cause unexpected behavior. Here are the **3 approaches I use**:

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$.pipe(takeUntil(this.destroy$)).subscribe();
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

```
<div *ngFor="let user of users$ | async">{{ user.name }}</div>
```

Handles subscribe and unsubscribe — zero risk of leaks.

```
private sub!: Subscription;

ngOnInit() {
  this.sub = this.service.data$.subscribe();
}

ngOnDestroy() {
  this.sub.unsubscribe();
}
```

**What you DON'T need to unsubscribe from: HTTP calls** (they auto-complete after response) and **ActivatedRoute observables** (Angular handles them). But for long-lived Observables like **BehaviorSubject streams, interval timers, and WebSockets** — always clean up.

**Real Example:** We once had a component with 3 unsubscribed observables — every time the user navigated to and from that page, **3 new subscriptions stacked up**. After 50 navigations, we had 150 active subscriptions causing lag. Adding `takeUntil` fixed it completely.

Proper cleanup is .

---

## Q4: What is Virtual Scrolling?

**A:** When you have a list of **thousands of items**, rendering all of them into the DOM at once makes the page **extremely slow and unresponsive**. Virtual scrolling solves this by **rendering only the items currently visible** in the viewport and **recycling DOM elements** as the user scrolls — so the DOM always contains just 20-30 elements, regardless of list size.

---

```
import { ScrollingModule } from '@angular/cdk/scrolling';
```

```
<!-- Template -->
<cdk-virtual-scroll-viewport itemSize="50" style="height: 400px;">
  <div *cdkVirtualFor="let item of items">
    {{ item.name }}
  </div>
</cdk-virtual-scroll-viewport>
```

**Real Example:** In my project, I had a transaction history page with **10,000+ records**. Without virtual scrolling, the page took **3+ seconds to render** and scrolling was laggy. After implementing `cdk-virtual-scroll-viewport`, it became **instant** — because only the ~20 visible rows exist in the DOM at any time. The difference was night and day.

Virtual scrolling is the without compromising user experience.

---

## Q5: What is Tree Shaking?

**A:** Tree shaking is a **build-time optimization** that automatically **removes unused code** from the final production bundle. Angular's build tools (webpack/esbuild) analyze the **import graph** to determine which functions, classes, and modules are actually used — everything else gets "shaken off" like dead leaves from a tree.

**How it works in practice:** If I install `lodash` (a 70KB library) but only use `_.debounce`, tree shaking removes all other lodash functions and only includes `debounce` in the bundle — saving significant bundle size.

- Use `providedIn: 'root'` for services — they're **tree-shakable by default**
- Use **ES module imports** (`import { debounce } from 'lodash-es'`), not CommonJS `require`
- Avoid wildcard imports (`import * as _ from 'lodash'` — prevents tree shaking)

**Real Example:** In my project, I switched from `import * as moment from 'moment'` to the lighter `date-fns` library with individual imports — combined with tree shaking, it **reduced our bundle by 200KB**.

Tree shaking ensures your production bundle contains — nothing more.

## Q6: What is Change Detection and how does it work internally?

**A:** Change Detection is Angular's mechanism to **sync the component's data model with the DOM**. Whenever something changes — a button click, an HTTP response, a timer — Angular walks the component tree, checks for changes, and updates the view where needed.

1. **Zone.js monkey-patches async APIs** — `setTimeout`, `addEventListener`, `Promise.then`, XHR, etc. Any async operation triggers Angular's zone, notifying it "something may have changed."
2. Angular then runs `ApplicationRef.tick()` — walks the **entire component tree** from root to leaves.
3. For each component, Angular compares current bound values against the previous snapshot.
4. If a binding changed → update the DOM.
5. In **dev mode**, Angular runs a **second pass** to catch `ExpressionChangedAfterItHasBeenCheckedError` — an expression that changes between the two passes indicates a bug.

| Feature | Default | OnPush |
|---|---|---|
| Checks every CD cycle | Yes | Only on specific triggers |
| Triggers | Any async event anywhere | `@Input` reference change, event in component, async pipe emission, `markForCheck()` |
| Performance | Slower for big trees | Much faster |

```
Async event (click / HTTP / timer)
        ↓
  Zone.js detects it
        ↓
  ApplicationRef.tick()
        ↓
  Traverse component tree (top → bottom)
        ↓
  For each component: compare bindings
        ↓
  Update DOM where values changed
```

**Real Example:** In my project, a dashboard with 40+ components was re-checking every component on **every keystroke** in a search box. By switching performance-heavy widgets (charts, tables) to `OnPush`, we reduced change detection time per cycle from **~25ms to ~6ms** — dramatically smoother interaction.

Angular is moving away from Zone.js. With **signals** (v16+), change detection becomes **fine-grained** — Angular knows exactly which components depend on which data and only updates those. Zoneless Angular (v18+ experimental) eliminates Zone.js entirely, relying on signals.

- *What causes* `ExpressionChangedAfterItHasBeenCheckedError` *?* — Modifying a bound value **inside lifecycle hooks that run during/after change detection** (like `ngAfterViewInit`). Fix with `setTimeout()`, `markForCheck()`, or refactoring the logic.
- *What does* `markForCheck()` *do?* — Marks the component (and its ancestors) as "needs check" on the next change detection cycle. Essential with `OnPush` when updating from outside normal triggers.

---

### Q7: What is Zone.js and what role does it play in Angular?

**A:** Zone.js is a library that **monkey-patches browser async APIs** — `setTimeout`, `setInterval`, `Promise`, `addEventListener`, XHR, etc. — to notify Angular when async work happens. Without Zone.js, Angular wouldn't know **when to run change detection**.

1. Creates an execution context called a **Zone** around the app
2. Wraps every async operation — so Angular gets a notification (`onMicrotaskEmpty`) when async work finishes
3. Triggers `ApplicationRef.tick()` → runs change detection → updates the DOM

**Without Zone.js** — you'd have to manually call `changeDetectorRef.detectChanges()` after every async operation. Zone.js is what makes Angular's "magic auto-updates" work.

```
Browser API (setTimeout)
        ↓  (Zone.js intercepts)
  Runs callback
        ↓  (Zone.js notifies Angular)
  Angular runs change detection
        ↓
  DOM updates
```

- Adds ~100KB to the bundle
- Patches every async API — occasional compatibility issues with third-party libs
- Triggers full-tree change detection on **every** async event — inefficient for large apps

Angular is moving toward removing Zone.js entirely. The modern stack — **Signals + standalone components** — lets Angular do change detection **only for components that actually changed**.

---

```
// main.ts — opt into zoneless experimental mode
bootstrapApplication(AppComponent, {
  providers: [provideExperimentalZonelessChangeDetection()]
});
```

**Real Example:** In my project, third-party WebSocket libraries sometimes ran **outside the Angular zone** — so UI updates didn't reflect until the user clicked something. Fixed by wrapping callbacks in `NgZone.run()` to re-enter the zone. Alternatively, `runOutsideAngular()` is useful when you want to run heavy background work **without triggering change detection** on every tick.

- *When would you use `NgZone.runOutsideAngular()`?* — For CPU-heavy work like animations, chart rendering, or high-frequency events (scroll/mousemove) — you don't want change detection running 60+ times per second.
- *Can you fully remove Zone.js?* — Yes, in Angular 18+ with `provideExperimentalZonelessChangeDetection()` and a signal-based app. Still experimental but production-ready in many apps.

---

## Q8: How do you reduce the bundle size of an Angular app?

**A:** Bundle size directly impacts **initial load time, Time-to-Interactive, and mobile performance**. Here's the full toolkit I use to keep bundles lean in production Angular apps.

— biggest win

```
{ path: 'admin', loadComponent: () => import('./admin/admin.component')
    .then(m => m.AdminComponent) }
```

Splits the bundle into chunks downloaded only when needed.

(Angular 17+) — load sections on demand

```
@defer (on viewport) {
  <app-heavy-chart />
}
```

Defers heavy components until they're actually needed.

— remove unused code

- Use `providedIn: 'root'` for services (tree-shakable)

- Use **ES module imports** (`import { debounce } from 'lodash-es'`) — not `import * as _ from 'lodash'`
- Keep `sideEffects: false` in library `package.json` where possible

| Heavy | Lighter |
|---|---|
| moment.js (~70KB) | date-fns (tree-shakable) or `Intl.DateTimeFormat` (built-in) |
| lodash | Native JS methods or lodash-es individual imports |
| rxjs full | Pipeable operators only — already tree-shaken with RxJS 7+ |

```
ng build --configuration=production
```

Enables AOT, minification, uglification, dead-code elimination, and `buildOptimizer`.

— prevent regressions

```
// angular.json
"budgets": [
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" }
]
```

Build fails if bundle grows beyond limits.

```
ng build --stats-json
npx webpack-bundle-analyzer dist/stats.json
# or for esbuild builds
npx source-map-explorer dist/**/*.js
```

Identifies which packages are bloating your bundle.

— enables more aggressive tree shaking of change detection code paths.

— reduces image payload with lazy loading and responsive sizing.

**10. Avoid polyfills for browsers you don't support** — check `polyfills.ts` and browserslist.

**Real Example:** In my project, combining **lazy loading + replacing moment with date-fns + removing unused lodash functions + `@defer` for charts** dropped our initial bundle from **2.1 MB to 620 KB** (gzipped). Lighthouse performance score jumped from 54 to 92.

- *What's the difference between `ng build` and `ng build --configuration=production`?* — Production enables AOT, optimization, hashing, no source maps. Development skips most of these

for faster rebuilds.

- *How do you identify the biggest offenders?* — Run bundle analyzer and look for large modules, duplicate dependencies, and accidentally-imported dev-only code.

---

# TOPIC 13: ANGULAR INTERVIEW SCENARIO QUESTIONS

### Q1: Tell me about a challenging problem you solved in Angular. (IMPORTANT)

**A:** "In my project, we had a **product search page** where users could search and filter from a list of 5000+ items. The problem was — every single keypress was triggering an **API call**, which caused the UI to lag, the server got hammered with unnecessary requests, and sometimes **older search results would overwrite newer ones** because of network timing.

I solved it using a combination of :

- `debounceTime(300)` — wait 300ms after the user stops typing before making the call
- `distinctUntilChanged()` — skip if the search term hasn't actually changed
- `switchMap` — automatically cancel the previous API call when a new one fires

This **reduced API calls by over 80%**, completely eliminated the stale results issue, and made the search feel **instant and responsive**. I also added **virtual scrolling** for the results list since it could return thousands of items — so the DOM only renders the visible rows.

The combination of transformed a sluggish feature into one of the smoothest parts of our application."

---

### Q2: How do you structure a large Angular application? (IMPORTANT)

**A:** I follow a **modular, feature-based structure** that keeps the codebase **organized, scalable, and easy to navigate** — even when the team grows:

```
src/
├── app/
│   ├── core/              → Singleton services (AuthService, interceptors, guards)
│   ├── shared/            → Reusable components, pipes, directives
│   ├── features/          → Feature modules (each lazy loaded independently)
│   │   ├── dashboard/
│   │   ├── users/
│   │   └── settings/
│   ├── models/            → TypeScript interfaces and type definitions
│   └── app-routing.module.ts
```

- **Core module** — contains singleton services (AuthService, API interceptors, error handlers). Imported **only in AppModule** — never in feature modules.
- **Shared module** — contains reusable UI components (buttons, modals, cards), pipes, and directives. Imported by **any feature module** that needs them.
- **Feature modules** — each feature is **self-contained and lazy loaded**. A new developer can understand a feature by looking at one folder.
- **Separation of concerns** — components handle only UI/presentation, services handle all logic and API communication.

**Real Example:** In my project with 15+ features and 4 developers, this structure allowed each developer to work on their feature module **independently** without merge conflicts or tight coupling. Adding a new feature was as simple as creating a new folder under `features/`.

This architecture ensures .

---

### Q3: How do you handle authentication in Angular? (IMPORTANT)

**A:** "I implement authentication using the **JWT (JSON Web Token) flow**, which is the most common approach for Angular SPAs:

**1. Login Flow** — User submits credentials → API validates and returns a **JWT token** → I store it in **localStorage** (or sessionStorage for stricter session control).

**2. Auth Interceptor** — A global HTTP interceptor that **automatically attaches** the token to every outgoing API request via `Authorization: Bearer <token>` header. No manual token handling in individual services.

**3. Auth Guard (CanActivate)** — Protects all private routes. Before allowing navigation, it checks if the user **has a valid, non-expired token**. If not → redirect to login page.

**4. Token Expiry Handling** — I decode the JWT to check expiry before critical API calls. If expired, either redirect to login or implement a **refresh token flow** to silently get a new token.

**5. Auth Service with BehaviorSubject** — Central service holding the current user state. Components subscribe to `currentUser$` to know the login status reactively — the header shows the username, guards check authentication, and the sidebar shows/hides menu items."

**Real Example:** In my project, this entire auth flow works seamlessly — login → interceptor adds token → guard protects routes → expired token triggers re-login. The user experience is smooth, and the code is **clean and centralized**.

- *Where do you store the token?* — `localStorage` for persistence across browser tabs. For better **XSS protection**, `httpOnly cookies` are ideal — but they require backend configuration and CSRF handling.

---

## Q4: How do you handle errors globally in your app?

**A:** "I implement a **layered error-handling strategy** that covers errors at every level — from specific component failures to unexpected global exceptions:

— Show context-specific error messages for specific feature failures (e.g., "Failed to load user list — please retry").

**2. Service level** — Use RxJS `catchError` to handle and transform API errors — return fallback data or user-friendly error messages.

**3. Global HTTP level (Error Interceptor)** — A single interceptor catches **all HTTP errors** and handles them by status:

- **401 Unauthorized** → Clear token, redirect to login
- **403 Forbidden** → Show "Access Denied" notification
- **500 Server Error** → Show generic "Something went wrong" toast
- **0 (Network Error)** → Show "Please check your internet connection"

**4. Unhandled Exceptions (Global ErrorHandler)** — Angular's `ErrorHandler` catches any **uncaught JavaScript errors** across the app:

```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any) {
    console.error('Global Error:', error);
    // Log to monitoring service (Sentry, LogRocket, etc.)
  }
}
```

**Real Example:** In my project, this layered approach ensures **no error goes unhandled** — HTTP errors show user-friendly messages, auth failures redirect cleanly, and any unexpected runtime error gets

logged to our monitoring dashboard for debugging.

---

### Q5: Explain a situation where you optimized performance. (IMPORTANT)

**A:** "We had a **dashboard page** that was our main landing page after login, and it was loading painfully slow — taking **4+ seconds** to become interactive. I investigated and found **three major bottlenecks**:

**Problem 1: Massive initial bundle** All 8 feature modules were loading upfront, even though the user only needed the dashboard. **Solution:** Implemented **lazy loading** for all feature modules except the dashboard. Initial load time dropped from **4s to 1.8s** immediately.

**Problem 2: Slow rendering of a large data table** A table with 2000+ transaction rows was causing the browser to struggle with DOM rendering and scrolling. **Solution:** Added **virtual scrolling** from Angular CDK and `trackBy` in `*ngFor`. The table now renders only visible rows (~25 at a time) — scrolling became **instant and smooth**.

**Problem 3: Excessive change detection** Multiple dashboard widgets (charts, counters, tables) were all using Default change detection, causing Angular to re-check **everything** on every user interaction. **Solution:** Switched performance-critical components to **OnPush change detection** strategy. Change detection cycles **reduced by over 60%**.

**Final Result:** Page load time went from **4 seconds to under 1.5 seconds**, scrolling was smooth, and the dashboard felt **responsive and professional**. This was the most impactful performance improvement I delivered."

---

### Q6: What is the difference between ViewChild and ContentChild?

**A:** Both are decorators for **querying elements** from the template, but they differ in **where** they look:

- **@ViewChild** — Accesses elements defined in the **component's own template** (its own HTML)
- **@ContentChild** — Accesses elements that are **projected into the component** from a parent via `<ng-content>`

```
// ViewChild – element in own template
@ViewChild('myInput') inputRef!: ElementRef;

// ContentChild – element projected from parent
@ContentChild('projectedItem') projectedItem!: ElementRef;
```

**Real Example:** In my project's reusable `CardComponent`, I use `@ContentChild` to access the projected header element and apply animations to it. For a chart component, I use `@ViewChild` to get

the canvas element defined in its own template.

**Simple rule:** Your own template → `@ViewChild`. Content coming from parent → `@ContentChild`.

---

## Q7: What are Standalone Components? (Angular 15+)

**A:** Standalone components are a **major simplification** introduced in Angular 15 — they **don't require NgModule** to function. Instead, they declare their own dependencies directly in the `@Component` decorator. This reduces boilerplate and makes components **truly self-contained**.

```
@Component({
  selector: 'app-hello',
  standalone: true,
  imports: [CommonModule, RouterModule],
  template: `<h1>Hello, {{ name }}!</h1>`
})
export class HelloComponent {
  name = 'World';
}
```

- **No more NgModule boilerplate** — each component manages its own imports
- **Easier to understand** — everything a component needs is declared right there
- **Better tree-shaking** — unused components are removed more efficiently
- This is the **official future direction** of Angular — new projects should prefer standalone

**Real Example:** In newer features of my project, I've started using standalone components. The developer experience is **noticeably smoother** — no more jumping between component and module files to add imports. Each component is a **self-contained unit** ready to use anywhere.

Standalone components are the — Angular is moving away from NgModules.

---

## Q8: What is the new Control Flow syntax in Angular 17+?

**A:** Angular 17 introduced **built-in control flow syntax** that replaces traditional structural directives with a **cleaner, more intuitive template syntax**. It's faster, requires no imports, and looks like modern template languages.

```
<!-- Old way with *ngIf -->
<div *ngIf="isLoggedIn; else loginTemplate">Welcome!</div>
<ng-template #loginTemplate>Please login</ng-template>

<!-- New way (Angular 17+) -->
@if (isLoggedIn) {
  <div>Welcome!</div>
} @else {
  <div>Please login</div>
}

<!-- Old *ngFor -->
<div *ngFor="let user of users; trackBy: trackById">{{ user.name }}</div>

<!-- New @for with built-in @empty -->
@for (user of users; track user.id) {
  <div>{{ user.name }}</div>
} @empty {
  <div>No users found</div>
}

<!-- New @switch -->
@switch (status) {
  @case ('active') { <span>Active</span> }
  @case ('inactive') { <span>Inactive</span> }
  @default { <span>Unknown</span> }
}
```

- **Cleaner, more readable** syntax — no more `ng-template` hacks for else blocks
- **Better performance** — optimized at the compiler level
- **Built-in `@empty` block** for `@for` — no need for separate `*ngIf` to check empty arrays
- **No CommonModule import needed** — these are built into the template compiler

**Real Example:** In my project, after migrating templates to the new syntax, code reviews became much easier — the control flow is **immediately readable** even for developers new to Angular.

The new control flow syntax is for Angular templates.

---

## Q9: How do you improve Angular app scalability?

**A:** Scalability means the app handles **growing teams, growing codebases, and growing traffic** without collapsing under its own weight. I treat scalability as a combination of **architecture, performance, and developer ergonomics**.

**1. Feature-based modular structure** — each feature self-contained under `features/`, independently lazy-loaded, with its own data-access, UI, and routing. Teams can work in parallel without stepping on each other.

**2. Enforce boundaries** — in Nx workspaces, lint rules prevent features from importing each other directly. All sharing goes through `shared/` or `core/` libs. Prevents the "spaghetti imports" that kill large codebases.

**3. Smart/Dumb component split** — dumb components become **reusable design-system pieces**. Smart components stay thin and feature-specific.

**4. Centralized state management** — service-based signal stores or NgRx for complex cases. A **single source of truth** prevents state drift when 5+ components manipulate shared data.

— for design system + utilities. Teams consume versioned packages, not raw source — enables independent upgrades.

- **Lazy loading +** `@defer` — initial bundle stays small no matter how many features exist
- **OnPush + Signals** — change detection scales with app size
- **Virtual scrolling** for large lists — rendering stays O(visible rows), not O(total rows)
- **SSR + hydration** — fast TTFB and good SEO at scale
- **Caching (shareReplay, HTTP interceptor, service worker)** — reduce backend load

- **Strict mode + strict templates** — compiler catches bugs before they reach PRs
- **Automated testing** — unit + integration tests at feature boundaries
- **CI pipelines** — linting, budgets, bundle analysis, e2e smoke tests on every PR
- `ng update` — automated Angular version upgrades keep the stack modern
- **Monitoring** — Sentry/Datadog for runtime errors; Lighthouse budgets for performance regressions

On my largest project — 18 developers, 400+ screens — the three things that made scaling manageable were:

1. **Nx monorepo** with enforced module boundaries (no circular deps)
2. **Lazy-loaded feature libs** with their own data-access layer
3. **Performance budgets** in CI that failed any PR adding >20KB to initial bundle

Without these, the app would have drowned in bundle bloat and merge conflicts within 6 months.

- [ ] Feature-first folder structure
- [ ] Every feature is lazy-loaded
- [ ] `OnPush` by default on new components
- [ ] Design system in a shared library
- [ ] Bundle budgets in `angular.json`
- [ ] Automated tests on critical paths

- ☐ Observability (errors + performance metrics) in production

- *When should you move to an Nx monorepo?* — Once you have 2+ apps sharing code, or 10+ developers working on one codebase. Nx adds tooling overhead but pays off massively at scale.
- *What's the #1 anti-pattern that hurts scalability?* — **Circular feature imports** — Feature A imports Feature B, which imports Feature A. Kills build times, breaks lazy loading, and makes the code impossible to reason about.

---

## QUICK REVISION CHEAT SHEET

| Topic | Key Points to Remember |
|---|---|
| **Compilation** | JIT = browser/runtime (dev), AOT = build time/production (faster, smaller) |
| **Components** | Building blocks with TS + HTML + CSS, tree structure |
| **Data Binding** | Interpolation `{{ }}`, Property `[]`, Event `()`, Two-way `[()]` |
| **Directives** | Structural ( `*ngIf` , `*ngFor` ) + Attribute ( `ngClass` , `ngStyle` ) |
| **Pipes** | Transform display data, async pipe auto-unsubscribes, prefer pure pipes |
| **Services** | Singleton with `providedIn:'root'` , share logic/data, keep components thin |
| **DI** | Angular auto-injects dependencies, hierarchical injectors, loosely coupled |
| **Routing** | RouterModule, Lazy Loading, Guards (CanActivate), Resolvers |
| **Forms** | Template-driven (simple) vs Reactive (complex), FormBuilder, Validators |
| **HTTP** | HttpClient returns Observables, Interceptors (auth token, error handling) |
| **RxJS** | `switchMap` (search), `debounceTime` , `takeUntil` (cleanup), `BehaviorSubject` |
| **Lifecycle** | `ngOnInit` (setup/API), `ngOnDestroy` (cleanup/unsubscribe) |
| **Performance** | Lazy load, OnPush, trackBy, virtual scroll, async pipe, avoid template functions |
| **State** | BehaviorSubject in services (lightweight), NgRx (complex), Signals (Angular 16+) |
| **Modern Angular** | Standalone components, Signals, new control flow ( `@if` , `@for` ), functional guards |

# INTERVIEW TIPS

1. **Always start with a direct answer** — then explain the why and how
2. **Connect every answer to your project** — "In my project, I used this for..."
3. **Don't bluff** — If you don't know, say "I haven't used it yet, but I understand the concept and can pick it up quickly"
4. **Use power keywords** — Interviewers listen for terms like "OnPush", "lazy loading", "switchMap", "BehaviorSubject", "tree-shakable", "standalone"
5. **Practice speaking out loud** — Read these answers aloud, not just in your head. Confidence comes from **practice, not memorization**
6. **Be ready to write code** — Interviewers may ask you to write interceptors, guards, reactive forms, services, or RxJS pipelines on the spot
7. **Show problem-solving mindset** — Don't just say what you used, explain **why** you chose that approach and what **alternatives** you considered

*Generated for Angular Interview Preparation | 2 Years Experience | Last Updated: April 2026*